LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title

CS5053NT Cloud Computing and the Internet of Things

50% Group Coursework

2025-26 Autumn

Student Name	LondonMet ID
Nischal Rai	24045948
Sarobar Pokhrel	24046061
Suranga Rai	24046119
Saphal Bohora	24046059
Nimesh Tamang	24045939

Team Name: .COM

Submitted To: Mr. Anish Pandit

Assessment Submission Date: Friday, January 16, 2026

Assessment Due Date: Friday, January 16, 2026

We confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

We would like to express my sincere thanks to my esteemed module leader, Mr. Anish Pandit and tutor Mr. Ishan Kafle, for their invaluable guidance, support, and encouragement throughout the development of this IoT project proposal. Their advice and insightful suggestions and constructive feedback played a vital role in assisting me with enhancing the depth. We are grateful for the time and effort that they devoted to helping me understand key concepts, help me tighten up my ideas and help me stay on the right track.

We really love the fact that they are ready to clarify doubts, suggest useful sources, and encourage me to become better at IoT and its skills and knowledge related to it. Their ongoing support has not only contributed to this proposal but has also provided strength to my whole learning experience. We are extremely thankful for their patience, mentorship, and commitment to student success.

Abstract

This project proposes a Bluetooth-controlled home automation system built around an ESP8266 /ESP32 and an HC-05 Bluetooth module. The system will allow a smartphone to toggle lights and fans via relays, control a water pump with automatic stop using tank-level sensors, detect gas leaks with an MQ-2 sensor and close the gas supply using a servo while sounding an alarm, and open a door automatically using a PIR motion sensor. The system emphasizes low-cost components, safety (gas shutoff), and local-control availability when Wi-Fi is not desired. Final Outputs include hardware prototype, Arduino/ESP firmware, a simple Bluetooth command protocol, and documentation for System Testing and Enhancement.

Table of Contents

Table of Figures

Table of Tables

1. Introduction

The concept of home automation has since become one of the major technological tendencies with more societies gradually making transition to smart environments to improve their safety, comfort, and conservation rates. In our group project, we highlight that technology is slowly transforming the manual household functions to automated systems of control under sensor, microcontroller, and wireless communication platform systems. The manual use of electrical appliances, including lights, fans, water pumps, etc., has become less convenient, particularly on the backdrop of the modern nature of our society, which requires convenience and distant usability.

The increasing necessity of affordable prototype home automation that assists with the energy saving, safety improvements, and making the daily functioning fewer complex sparks the interest of this project. Our studies and research indicate that incidents as gas leaks, unattended electrical appliances and uncontrolled water pumps contribute to household accidents, energy wastage, and equipment damage. Additionally, elderly individuals and people with mobility limitations may face difficulties performing repetitive manual tasks, such as operating switches and monitoring gas stoves.

Integration of Bluetooth/Wi-Fi connectivity, ESP32 microcontrollers, relay circuits, motion sensor, water level sensor, and gas leakage detection creates smart, safe, and user-friendly home automation system. The proposed system architecture aims to address issue related to energy wastage, unattended appliances, and household safety concerns.

1.1 Current Scenario

Although smart home technologies have advanced significantly in the recent year but large numbers of households in Nepal particularly in developing regions still rely on the manual operations of electrical appliances, water pumps and gas systems. Daily activities such as switching lights and fans, controlling water pumps and monitoring gas usage are performed without any form of automation or real time monitoring. This traditional approach not only increase physical effort but also contribute to inefficient energy usage and higher safety risks within the household.

Limited awareness, affordability concerns and lack of technical exposure are major factors that restrict the adoption of home automation systems. As result, many users remain unaware of energy wastage caused by unattended appliances or delayed response to hazardous situations. Recent safety reports and studies have highlighted an increase in domestic accidents related to gas leakage and electrical faults. This event occurs due to absence of automated detection and alert mechanisms, emphasizing the need for intelligent and preventive systems.

Moreover, elderly individuals and people with mobility impairments face additional challenges in manually operating household device and responding quickly during emergencies. This scenario demonstrates clear requirements for an affordable, easy to use and reliable home automation system that can reduce energy wastage, enhance safety and improve over all living standards.

1.2 Problem Statement and Project as a solution

Manual operation of household electrical appliances involves several limitations including inconvenience, inefficient energy usage, delayed emergency response and increased safety risks. Incidents such as gas leakage, unattended appliances and uncontrolled water pump operation may lead to serious accidents, energy loss and property damage, particularly affecting people with mobility limitations and elderly individuals.

To Address these issues, this project proposes the development of an affordable WiFi based home automation system. The system enables automated and remote control of household appliances such as fans, lighting, door access mechanisms, water pump and gas safety units. By integrating sensors, micro controller-based control and wireless communication, the

proposed solution enhances the household safety, reduces energy wastage and improves overall user experience and convenience.

1.3 Manual System vs Smart System

In a manual type of household system, users physically operate electrical appliances, water pumps and safety mechanisms which has traditionally resulted in inefficiency like wastage of energy, slow reaction to dangerous conditions and a greater physical effort. The environment needs the presence of human beings at all times to do the duties such as switching lights, checking water levels, or it may detect gas leakage, and the chances of accidents are high when appliances are not monitored. A smart home automation system, conversely, is capable of automatically and remotely controlling house devices with the use of sensors, microcontrollers, and wireless communications. Smart systems may also react instantly to environmental variations, like turning of gas when it is leaking or turning off of water pumps when the tanks are full, which improves safety, convenience, and energy efficiency. This manual-to-smart system is an important enhancement of the reliability, loss of human error, and modern living needs, particularly among the elderly and the mobility-disadvantaged people.

1.4 Aims and Objectives

1.4.1 Aims

To build a secured and affordable prototype of a Bluetooth-controlled home automation system using the ESP32 microcontroller. The prototype includes remote switching, automatic water tank regulation, gas leakage protection, and motion-based door control to improve household safety and convenience.

1.4.2 Objectives

- For a smartphone to control electrical appliances by wife module communication protocol.
- Implement automatic pump control, responding to the reading of the tank level sensor, with the task of stopping the operation when the tank is full so as not to cause tank depletion.
- Gas leak detection: - Detect gas leakage using the MQ-2 sensor, turn on the buzzer and servo to turn the gas off, and store the information locally

- Implement a PIR - based automatic door opening system that will facilitate convenience and support elderly/mobility restricted people
- Test and verify amongst the system regarding response time, reliability, safety accuracy as well as error handling under a few scenarios.

2 Background

2.1 System Overview

The architecture of the smart home automation is developed on the principles of Internet of Things (IoT) to enhance the household safety, convenience, and energy efficiency. The system is used to displace manual control of household appliances by automated and remotely controlled functions which cut down on human efforts and reduced safety hazards. The system offers a relatively affordable solution that can be used in residential settings using affordable hardware and wireless communication technologies.

The ESP32 microcontroller is the main control system of the system. It communicates with other sensors like MQ-2 gas sensor that detects gas leaks, an IR sensor that detects motion or proximity and an ultrasonic sensor that detects the level of water. These sensors constantly gather environmental information which is run through the ESP32 to make real-time decisions according to a set of logic.

The system has output devices based on sensor readings such as a servo motor, a relays module, a buzzer and an LED. The high-voltage appliances (lights, fans, and a submersible water pump) are operated safely by relay modules whereas the servo motor is applied to the automated control of a door. The buzzer and LED give auditory and visual signals when hazardous conditions are attained like gas leakage.

The system takes power through various sources with USB powering the ESP32 through a laptop, low-voltage components connected to the breadboard through power rails and outside 9V supplies powering relays and motors. Appliances can be monitored and controlled by users using a web-based interface or local device via wireless communication by Wi-Fi (and Bluetooth where available).

All in all, the system provides a single smart home platform, which uses automation, safety monitoring, and remote control. The scalable and modular design can be enhanced in the future with the addition of cloud integration, support of mobile applications, and data analytics.

2.2 Design Diagram

2.2.1 Block Diagram

The block diagram gives a simple overview of the system. It uses blocks to show the main components of the system and uses arrows to show how the components are linked to each other. This diagram makes it easier to understand the overall working of the system without going or showing technical details.

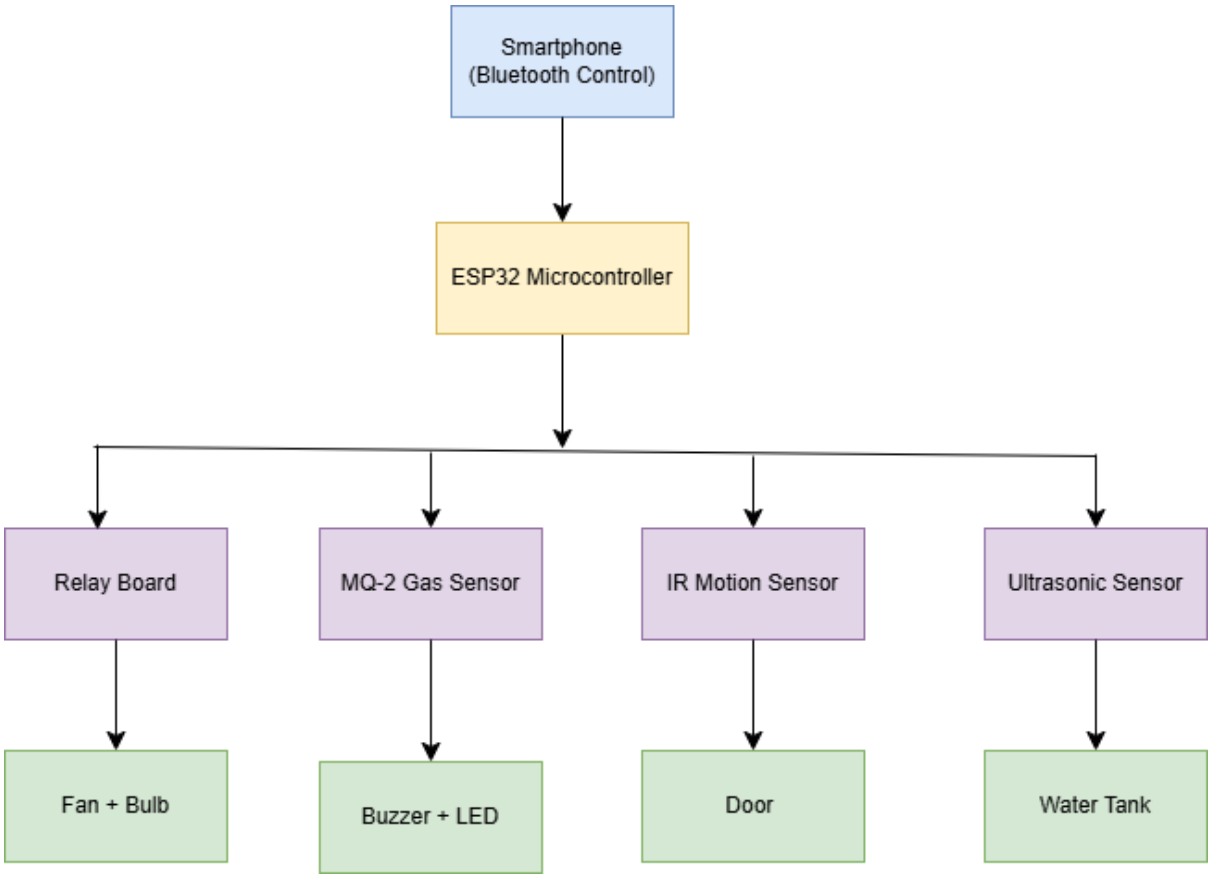

Figure 1: Block Diagram

2.2.2 System Architecture

The system architecture is a representation that shows the overall structure of the system. It shows the connection between sensors, microcontroller, and the output devices and work together in a system. This diagram makes easier to see how all the components of the system interact and connect.

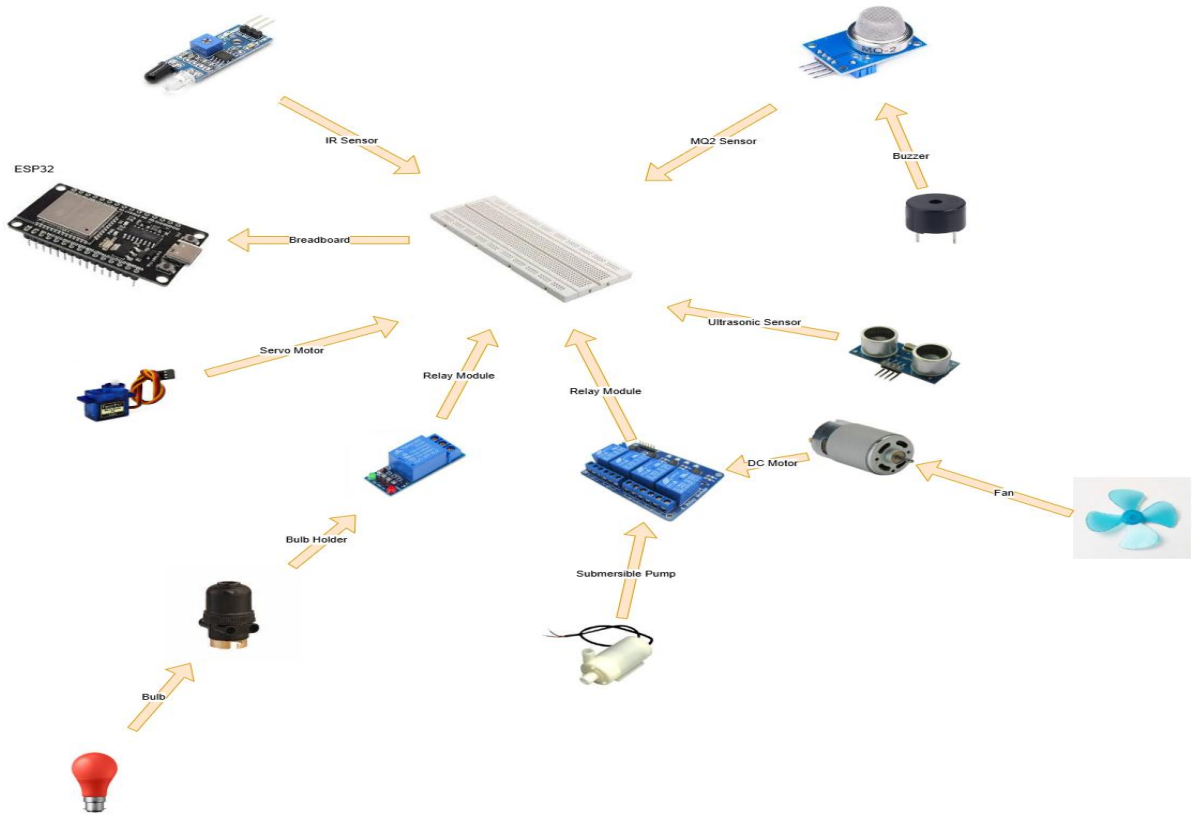

Figure 2: System Architecture

2.2.3 Flow Chart

The flow chart is a diagram that explains the step-by-step working of the system with logic. It can be followed in its logical sequence of operations beginning with input and going through every decision to the ultimate output. Using flowchart, we can easily see how the system makes decisions and what occurs at every stage. It assists in clarifying the way the system reacts to various inputs in a way that anyone can be able to trace the process to the very end without confusion.

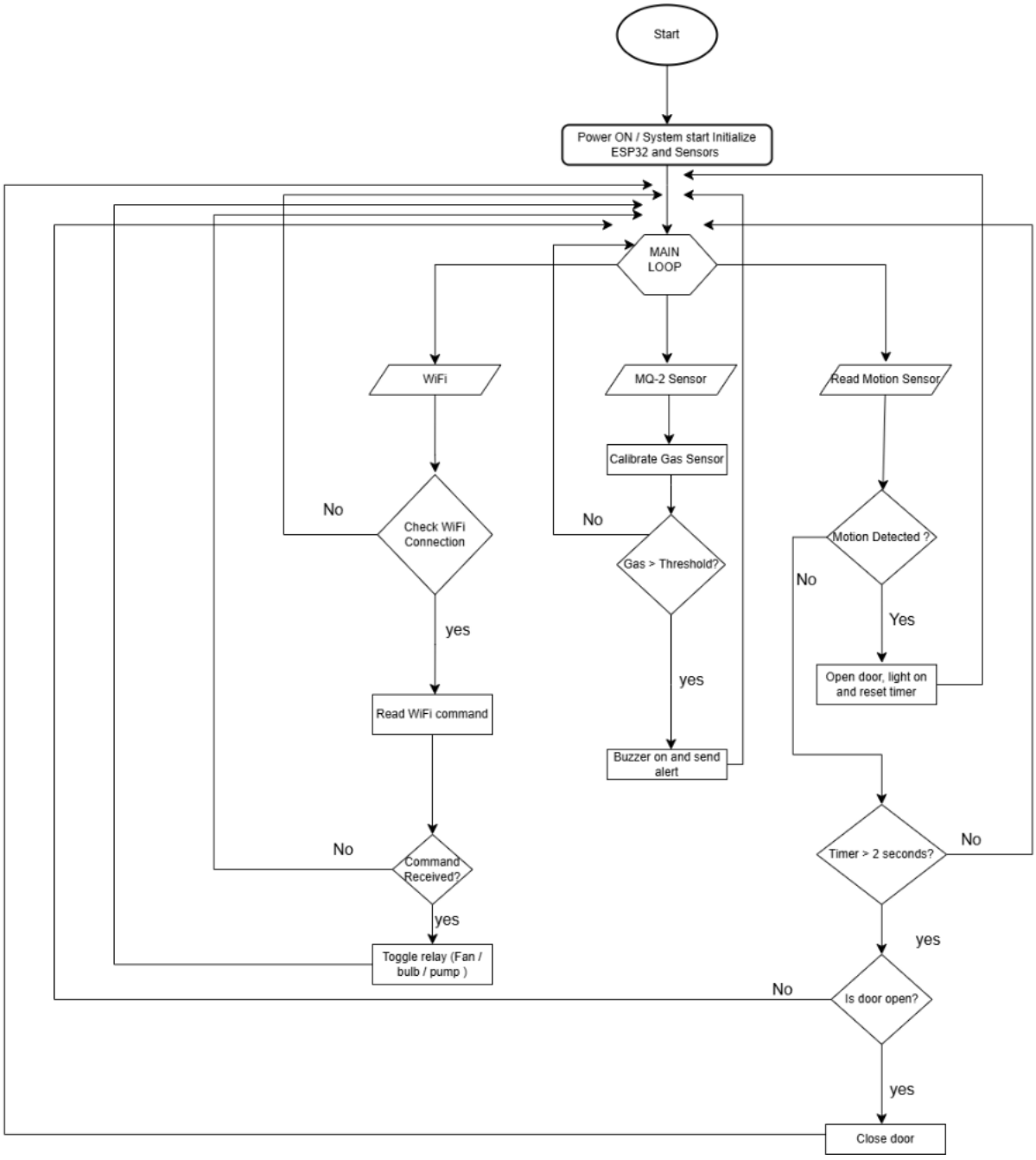

Figure 3 Flow Chart

2.2.4 Circuit Diagrams

The circuit diagram is a diagram which shows the connection of ESP32 microcontroller to all the sensors, actuators, and other modules in the system. It is concerned with the actual, physical connection of the components, to ensure that we can see how all the connection is made This diagram helps in looking how the components interact and make the system operate safely and correctly. Anyone can follow the connections according to this diagram.

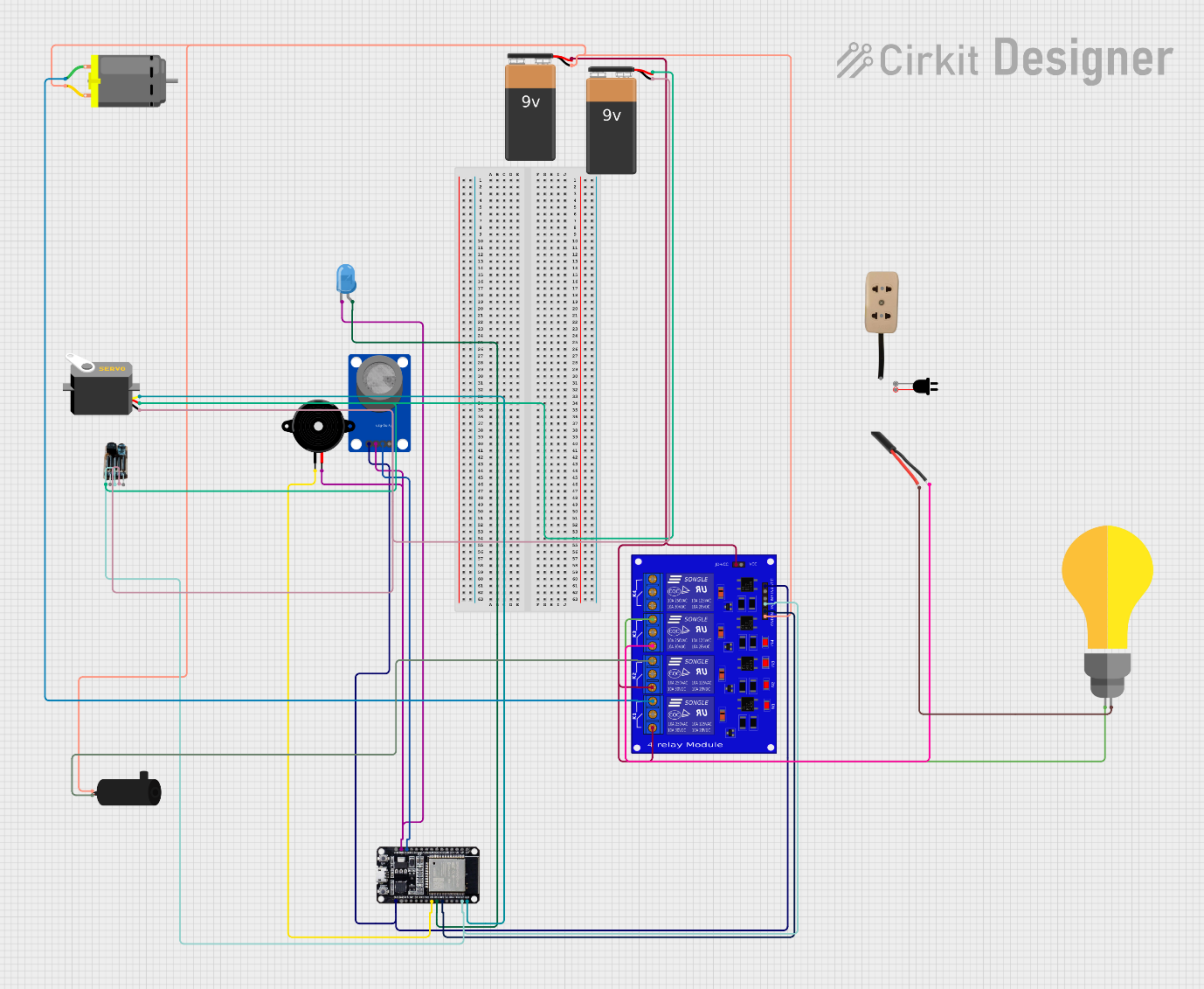

Figure 4: Circuit Diagrams

2.2.5 Schematic Diagram

The schematics diagram is a diagram that shows the system using electronic symbols instead of real components. It focuses on the logical connections of the electronic parts and not on physically positioning parts. The diagram is very useful for understanding the design of the electronics since it is easier to follow the circuit flow, check how components are connected, and how the system works without having to look at the actual hardware.

2.3 Requirement Analysis

2.3.1 Hardware Requirements

2.3.1.1 **ESP32:**

The ESP32 is the main processor of the system. It is an efficient, inexpensive microcontroller that has embedded Wi-Fi and Bluetooth features that are useful in the IoT applications. ESP32 Indicates the sensor data, control logic, and provides a web server and communicates wirelessly with the user devices in this project. It has got low power consumption, and high processing capability, which can be used to be reliable in real-time performance of multiple sensors and actuators at a time.

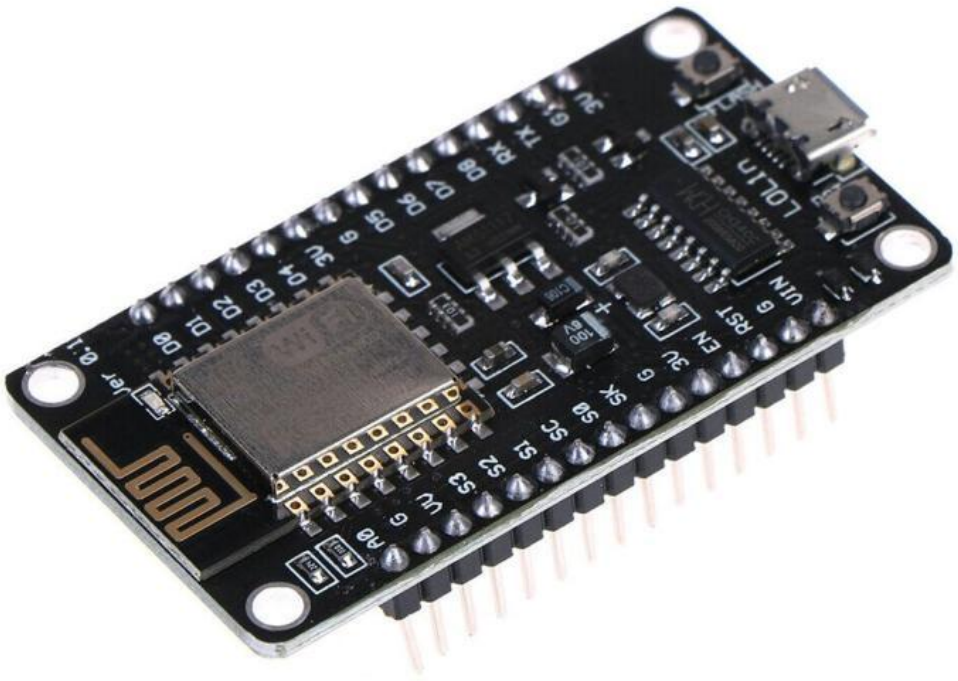

Figure 5 ESP 32

2.3.1.2 **2/4 channel Relay board:**

High-voltage household items like bulbs, fans and water pumps are controlled using High-voltage Battery signals sent to the ESP32 by means of relay modules. The relay is an electrically isolated switch that is safe and provides the opportunity to have the appliances turned on or off automatically or remotely by the microcontroller. This makes it possible to add any traditional electrical device to a smart automation system.

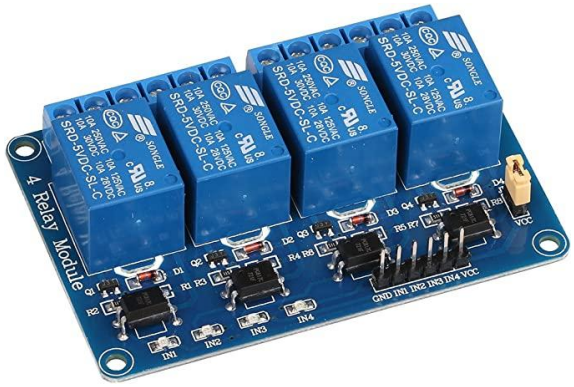

Figure 6 4 channel (or more) relay board

2.3.1.3 **MQ 2 gas sensor:**

MQ-2 gas sensor is applied in detecting flammable gases, including LPG, methane, smoke, and carbon monoxide. It gives analog output values which are different according to gas concentration. Under this system, the sensor constantly checks the quality of the air, and in case the gas levels become higher than a fixed limit, the sensor raises alarms and safety measures. This element is a key to improving the level of safety in the household.

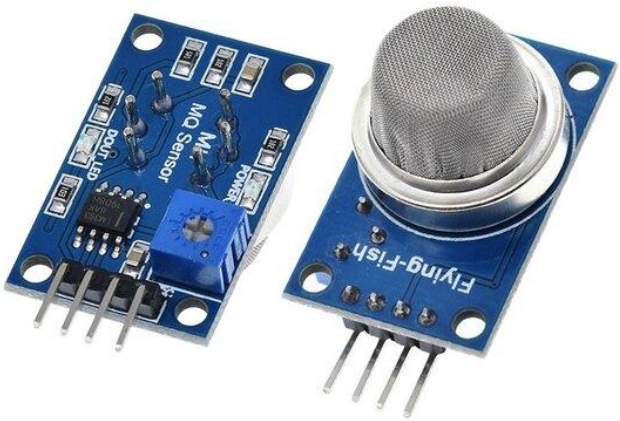

Figure 7 MQ 2 gas sensor

2.3.1.4 **Ultrasonic sensor:**

The ultrasonic sensor works based on a distance measurement method whereby it sends sound waves in the high frequencies and determines the time the sound echo takes before bouncing back. In the present project, it is applied to measure the water level in a tank. Depending on the distance measured, the system will automatically operate the water pump, this will help avoid overflowing of the water and waste of water. The sensor gives precise and real time distance measurements.

Figure 8 Ultrasonic sensor

2.3.1.5 **Submersible pump:**

The simulation of water pumping in the real world is carried out with the help of the submersible pump. The pump is an automatic control system with the ultrasonic sensor readings controlled by the relay or a manual control system by the web interface. This element proves to be practically automated in water management systems.

Figure 9 Submersible pump

2.3.1.6 **SG90 /Metal Servo:**

The door is controlled to move by the servo motor which rotates to the angles directed by sensor input. It can be controlled with PWM signals of the ESP32, which means that it can be controlled with a high degree of angularity. In this project, the servo motor was used to automatically open and close the door when it sensed motion or detected an object, and this enhanced access and convenience.

Figure 10 SG90 /metal servo

2.3.1.7 **PIR motion sensor:**

The door is controlled to move by the servo motor which rotates to the angles directed by sensor input. It can be controlled with PWM signals of the ESP32, which means that it can be controlled with a high degree of angularity. In this project, the servo motor was used to automatically open and close the door when it sensed motion or detected an object, and this enhanced access and convenience.

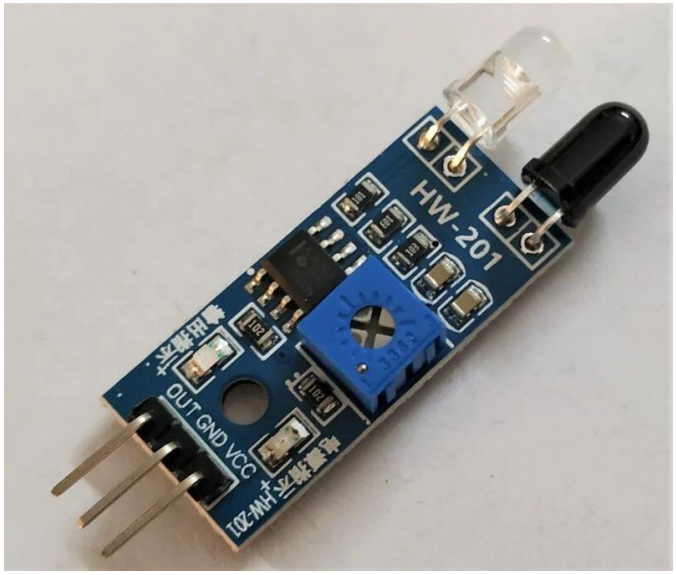

Figure 11 IR for motion sensor

2.3.1.8 **Buzzer:**

The buzzer will give audible signals in case of any dangerous instances like leakage of gases. It makes the system more responsive and safety-focused through the immediate warning of the occupants of danger and thus more user-aware.

Figure 12 Buzzer

2.3.1.9 **LED:**

The visual indicators are LEDs that are used to represent the state of the system (e.g., motion detection, gas detection, or activation of a relay). They give immediate visual response which helps in debugging and monitoring the system.

Figure 13 LED

2.3.1.10 **. Jumper Wire:**

It is used to connect components on breadboard for testing and building circuits.

Figure 14 Jumper Wire

2.3.1.11 **Bulb holder:**

The bulb holder is a secure holder that allows the bulb to securely be held and is able to give safe electrical connection to the power supply. It has good mounting and electrical safety when combining lighting circuit with the automation system.

Figure 15 Bulb holder

2.3.1.12 **9v Battery:**

It is common, small and rectangular power source delivering 9 volts of power, known for its snap connector and used in devices like toys, smoke detectors and wireless electronics where we used this to give power for a Dc motor and submersible pump.

Figure 16 9v Battery

2.3.1.13 **Plasti Fan Blade :**

Fan used in DC motor are either brushed or brushless. Brushed DC motor is traditional, cost effective and often used in simple fans for DIY projects, toys.

Figure 17 Fan Blade

2.3.1.14 **DC motor:**

DC motor converts direct current (DC) electrical energy into mechanical rotational energy, using magnetic fields to create force, making things spins and used in everything from toys to robots and we demenostrate the smart fan control is demonstrated using the DC motor and fan. The fan is operated on a relay and Wi-Fi interface and can be switched on or off remotely, which is a real-world application of appliance automation.

Figure 18 DC motor

2.3.1.15 **Bulb:**

The lighting appliance used in the home automation system is the bulb, which is employed to show smart lighting control. It is connected to the ESP32 by a relay which is switched ON and OFF demonstrating successful remote and automated control of **household lighting.**

Figure 19 Bulb

2.3.1.16 **. Breadboard:**

Breadboards are a construction base used to build semi-permanent prototypes of electronic circuits and do not require soldering or destruction of tracks and are reuseable.

Figure 20 Breadboard

2.3.2 Software Components

2.3.2.1 **Arduino IDE:**

It is an open-source software platform used for writing, compiling and uploading code to Arduino boards. It provides user friendly interface for coding in languages based on C/C++ specifically designed for interacting with Arduino hardware.

Figure 21 Arduino IDE

2.3.2.2 **Tinker Cad:**

Tinkercad is a free, web-based 3D modeling and computer-aided design (CAD) platform developed by Autodesk, designed to make 3D design, electronics, and coding accessible to beginners and educators. The platform uses a simplified constructive solid geometry approach, allowing users to create models by dragging and dropping primitive shapes—such as cubes, spheres, and cylinders—onto a workplane and combining them as solids or holes to form complex designs.

Figure 22 TinkerCad

3 Development

3.1 Resource Allocation

Allocation of resources to this project was done with a lot of care so that the development and implementation of the smart home automation system can be done effectively. The resources were classified as, hardware resources, software resources, and human resources.

The hardware resources were the ESP32 microcontroller, MQ-2 gas sensor, IR sensor, ultrasonic sensor, relay modules, servo motor, buzzer, LED, breadboard, connecting wires, power supplies (USB and 9V batteries), and controlled appliances (light, fan, and water pump). These elements were chosen depending on their availability, cost-effectiveness and compatibility with the system design.

The software resources included the Esp32 IDE to write software programs, necessary ESP32 libraries, serial monitor to debug the program, and simple documentation software to prepare the report. These computer programs aided in writing code, testing and monitoring the system.

In general, the efficient allocation of resources contributed to performance optimization as well as minimizing the development time and efficiency in reaching the project goals.

3.2 System Development

Phase 1: Connecting ESP 32 to Laptop

- Firstly ESP 32 was connected to breadboard, VIN/5V to +ve and gnd to -ve in breadboard.
- Then, ESP 32 was connected to laptop by using USB cable.

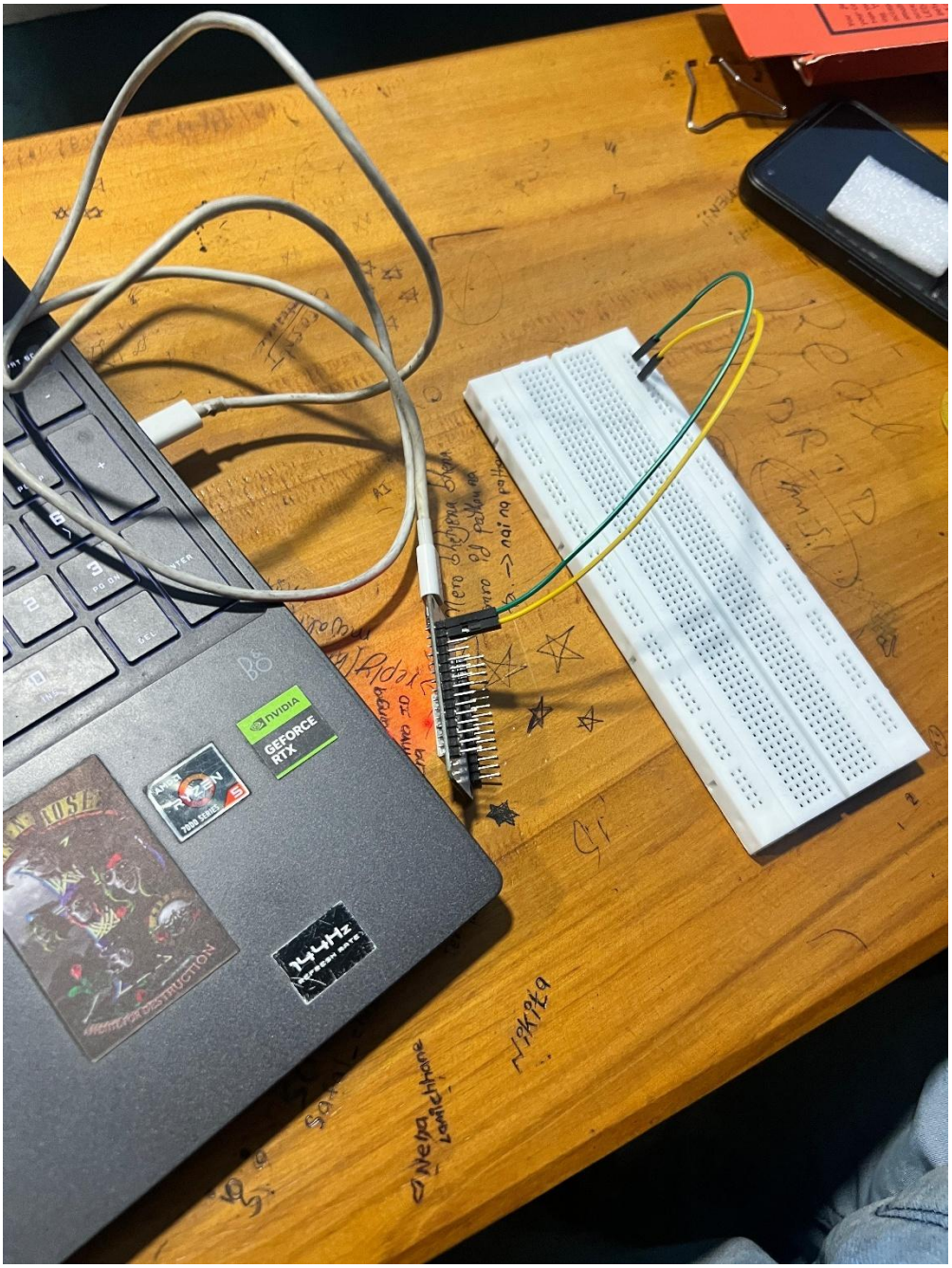

Figure 23: Connected ESP 32 to Laptop

Phase 2: Connecting ESP 32 to PIR Sensor for motion detection

- Pir sensor was connected to Esp by connecting: pir vcc to 5v/vin, pir gnd to gnd in esp, pir out to gpio 27.

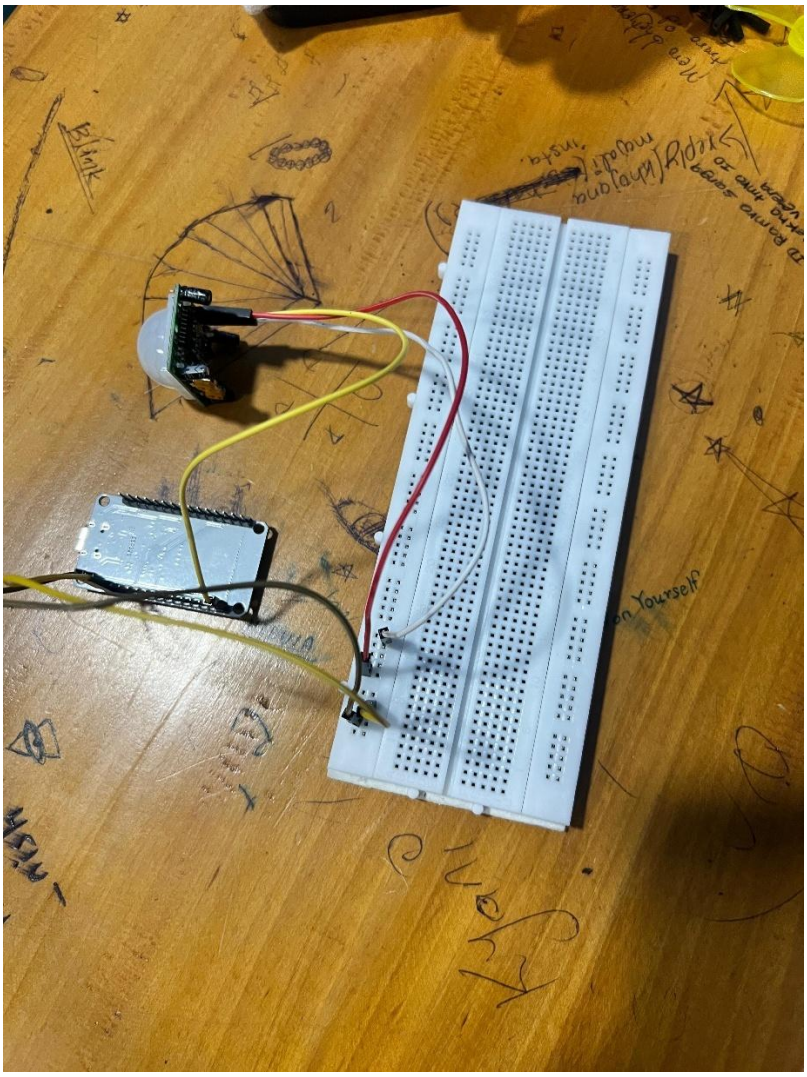

Figure 24: Connected PIR sensor to ESP 32

Phase 3: Connecting ESP 32 to MQ-2 for gas detection

- Mq-2 sensor was connected to Esp by connecting: mq-2 vcc to 5v/vin, mq-2 gnd to gnd in esp, pir out to gpio 34.

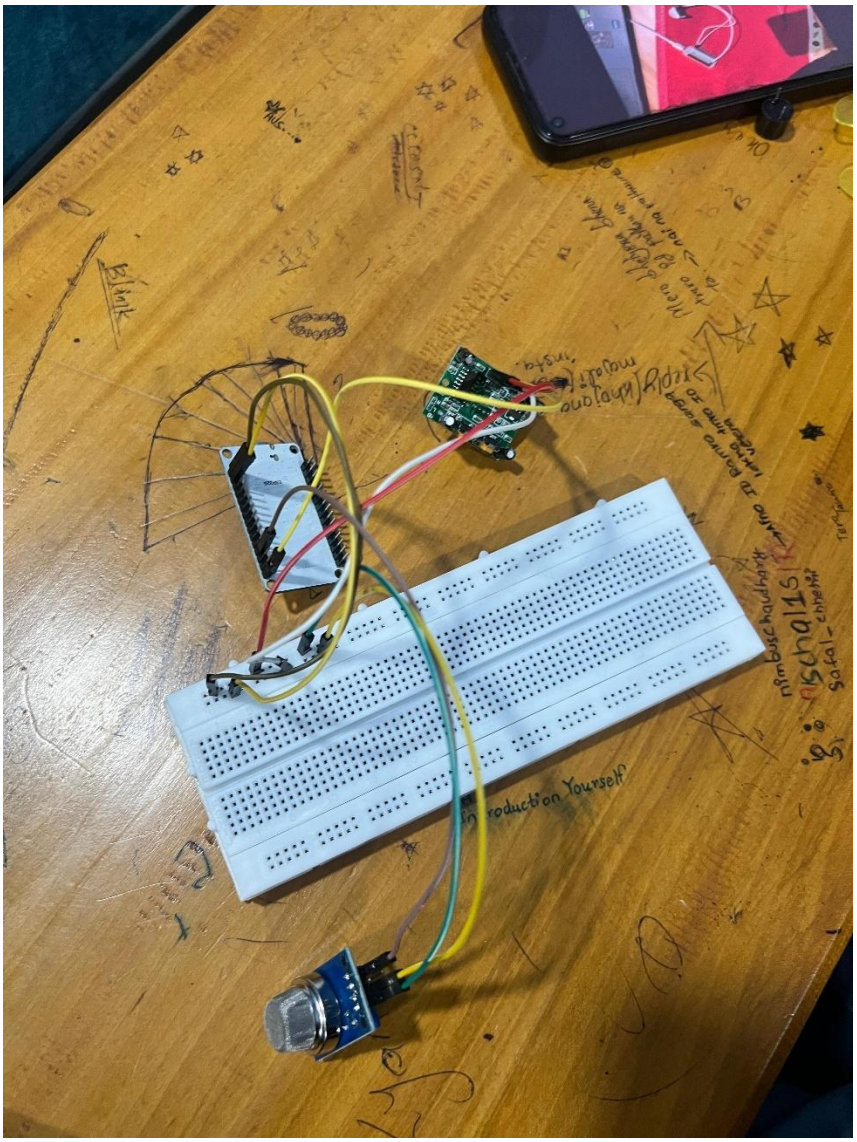

Figure 25: Connected MQ-2 to ESP 32

Phase 4: Connecting ultra sonic sensor to ESP 32 for object detection

- Ultra sonic sensor was connected to Esp by connecting: ultra sonic VCC to 5v/vin, ultra sonic GND to GND in ESP, ultra sonic trig to GPIO 05, ultra sonic echo to GPIO 18.

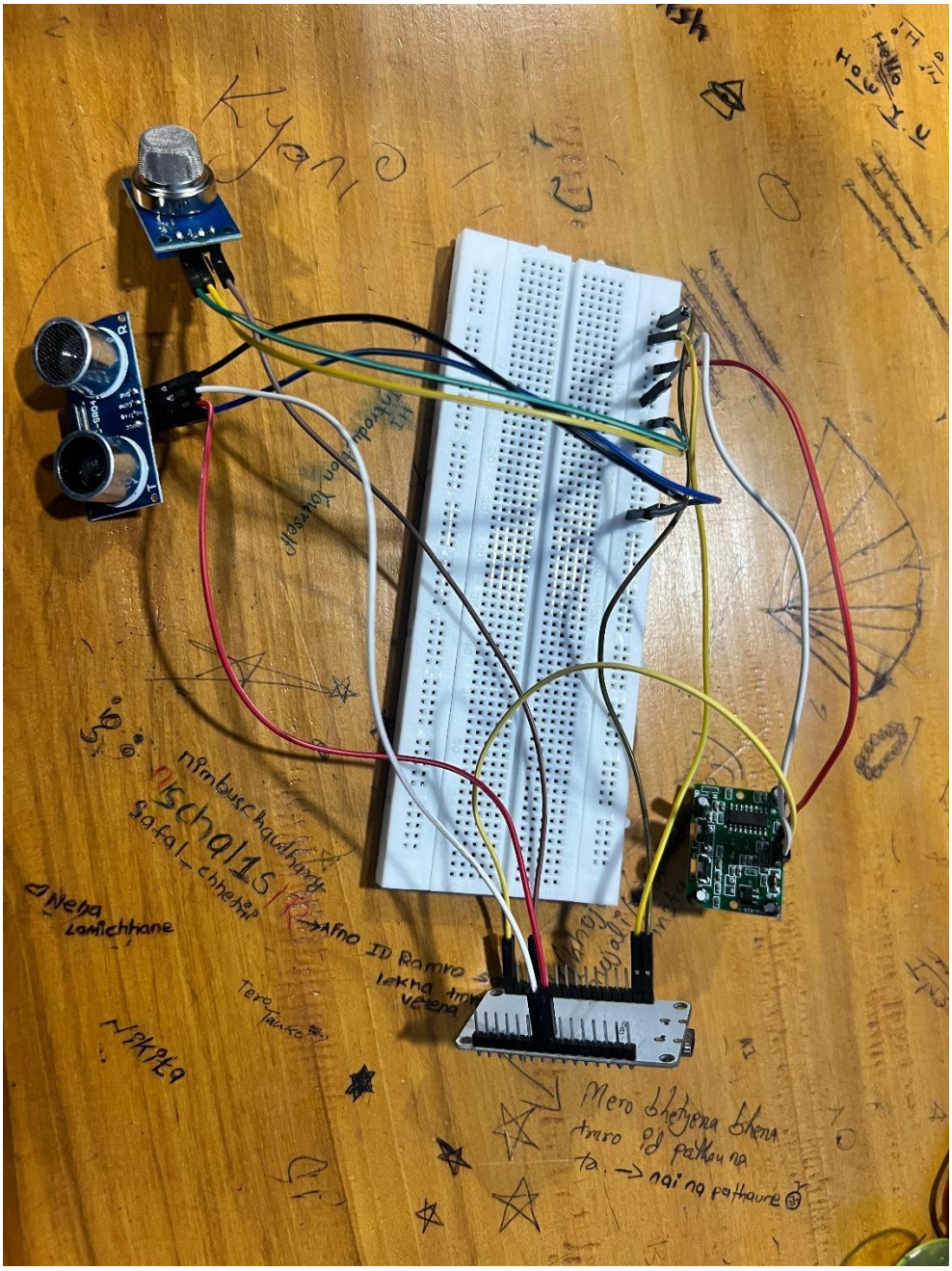

Figure 26: Connected Ultra Sonic Sensor to ESP 32

Phase 5: Connecting servo motor to ESP 32 for door automation

- Servo motor was connected to Esp by connecting: Servo motor vcc to 5v/vin, Servo motor gnd to gnd in esp, Servo motor signal to GPIO 19.

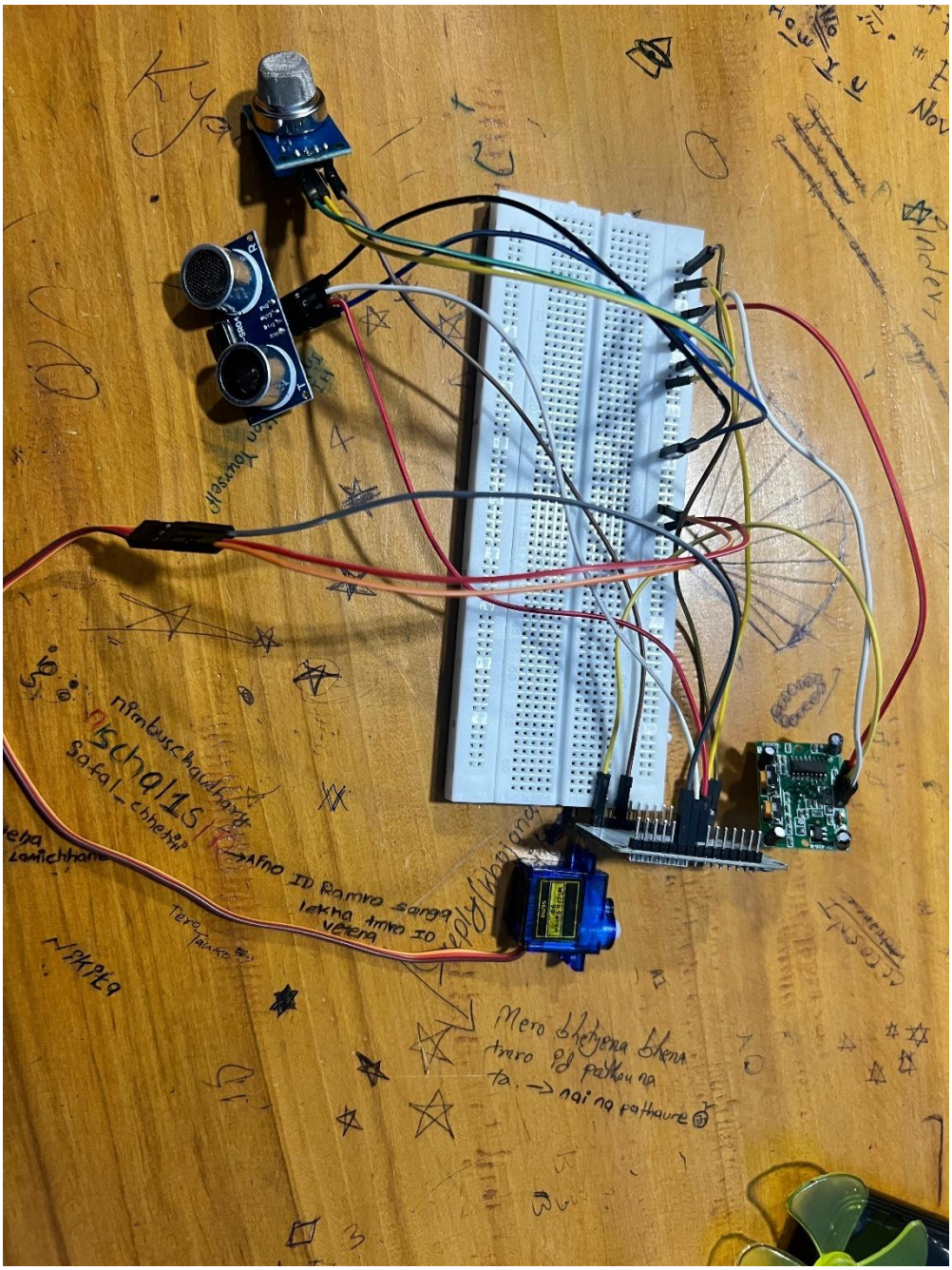

Figure 27: Connected Servo Motor to ESP 32

3.3 Cloud Integration

Cloud integration enables the remote operation and control of the smart-home system over the internet in the form of a web interface. We selected the ESP32 microcontroller in this project as it has inbuilt Wi-Fi which will enable us to connect the whole house directly to a cloud-based MQTT broker. The sensor data is continuously updating the cloud server with sensor information like the values of gases, motion status, and device state.

The web-based smart-home dashboard exists on the internet and is subscribing to the cloud data and presents real-time system status where the bulb, fan, pump, motion sensor, gas sensor and door servo status are displayed. As a result of this dashboard, it is possible to remotely operate appliances, sending commands to the cloud, which are then received by the ESP32 via MQTT communication and implemented immediately.

This cloud technology allows users to turn and monitor their appliances in the home, anywhere in the world without necessarily being there physically, through a web browser. It improves convenience, scalability, and real time accessibility and exhibits a practical internet of things implementation with the use of cloud.

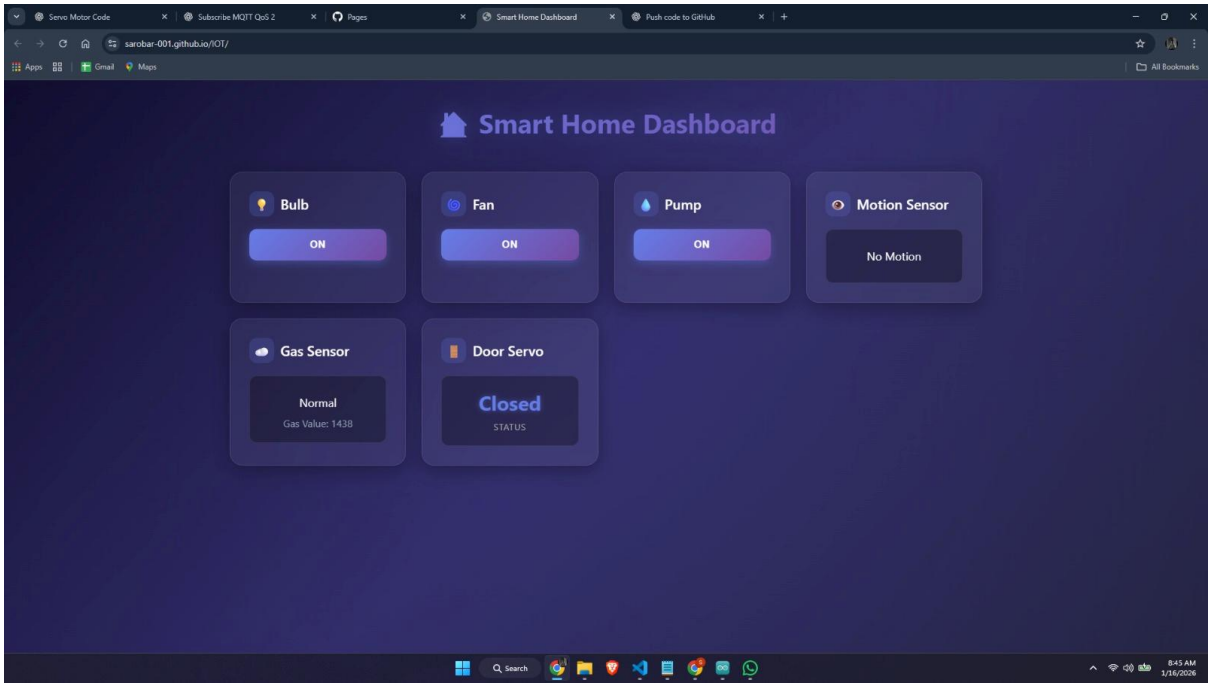

Figure 28:Cloud Integration Frontend

4 Result and Findings

4.1 Test 1: PIR Sensor

Table 1 Test of PIR

Test 1	**PIR Sensor**
Objective	To test if PIR (motion) sensor can detect human movement and automatically control servo motor to open/close door
Action	PIR Sensor continuously monitors for motion. When it detects movement, ESP32 signals servo motor to rotate from 0 to 90 Degree (door open). Led on pin 12 also light up. After 2 seconds of no motion, servo returns to 0 degree (door close).
Expected Result	When someone moves near sensor: • Servo should rotate to 90 degrees to open • LED should turn ON • Serial monitor shows "Motion detected →Door opening • When no motion for 2 seconds: • Servo return to 0 degrees to close • Serial monitor shows "No motion detected → Door closing
Actual Result	The system worked perfectly as shown in the serial monitor: • No motion detected → door closing • Servo angle to 0 degree • Motion detected → door opening • Servo angle to 90 degree • No motion detected → door closing • Servo angle to 0 degree The cycle repeated smoothly. The blue LED in image is glowing showing system is active.
Conclusion	• PIR sensor detects motion accurately • Servo motor moves precisely to 0 degree and 90 degrees

	• LED indicator works correctly • 2-second delay (CLOSE_DELAY_MS = 2000) • functions as programmed • System is reliable for automatic door control applications

4.1.1 Result

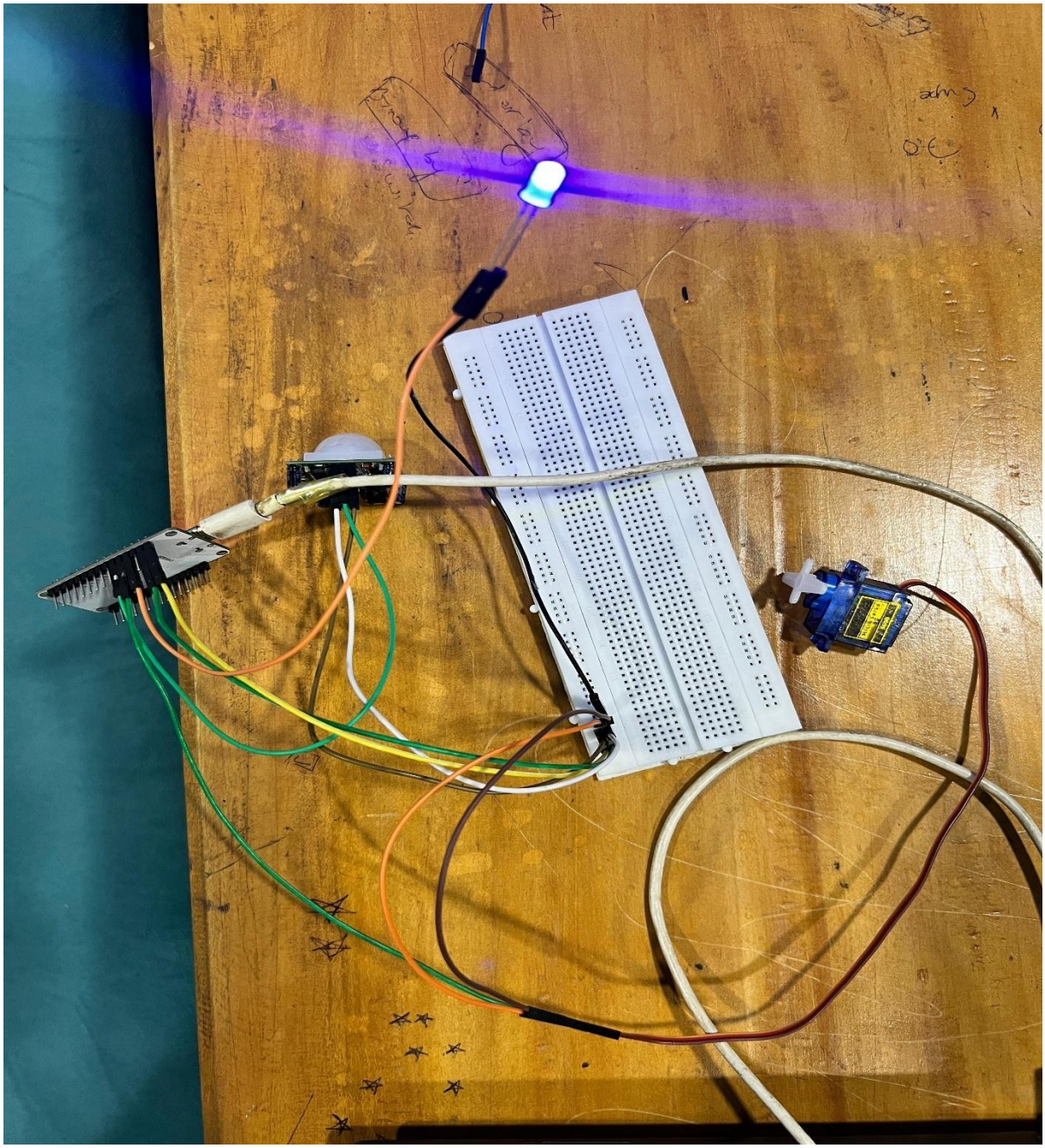

Figure 29 PIR Sensor

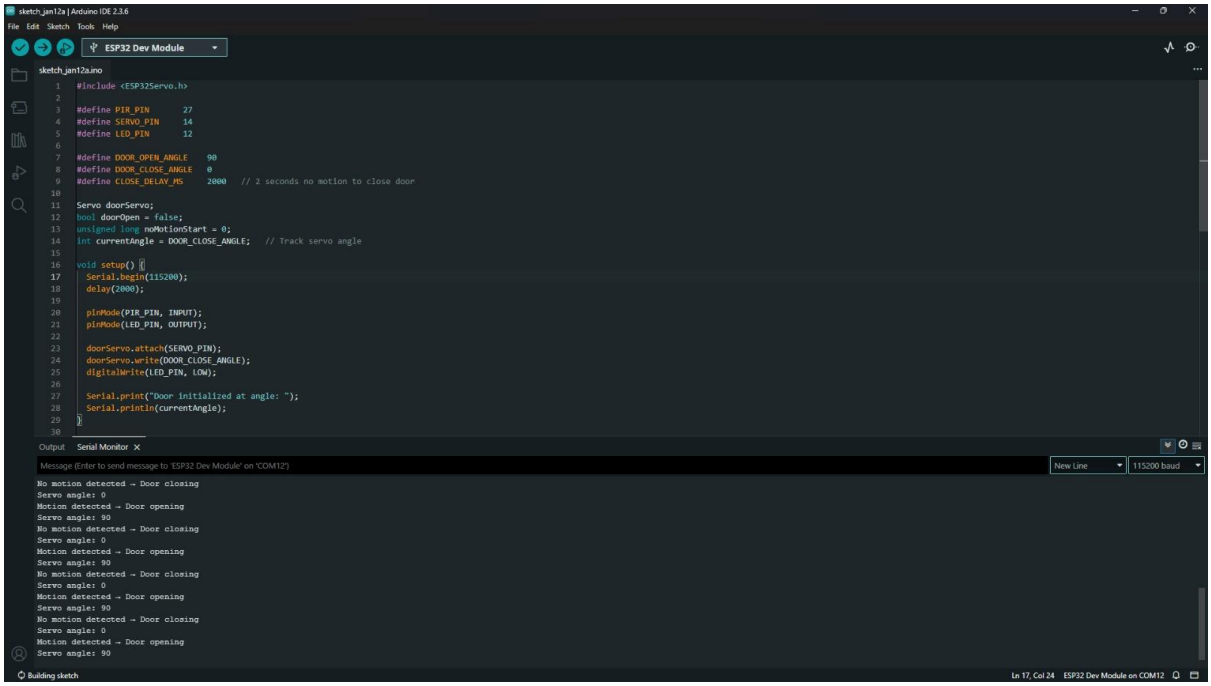

Figure 30 PIR Output

4.2 Test 2: Gas Sensor

Table 2 Test of Gas Sensor

Test 2	**Gas Sensor**
Objective	To test if MQ-2 gas sensor can detect harmful gases (like LPG, smoke) and activate warning systems (buzzer and LED)
Action	MQ-2 gas sensor first calibrates itself by taking 50 readings in clean air to establish a baseline. Then it continuously monitors air quality. When gas levels exceed the threshold, buzzer sounds and the LED turns ON to alert users
Expected Result	During startup: • "Calibrating... Keep sensor in clean air" message appears • System takes 50 readings and calculates baseline During operation: • When gas detected: buzzer beeps, LED glows, "Gas detected!" appears with sensor values • In clean air: no alerts
Actual Result	Calibration completed successfully as shown in code (lines 17-23). The serial monitors displayed: • "Gas detected!" (multiple times) • "Sensor Value: 2640"- "Sensor Value: 2672" • "Sensor Value: 2634"- "Sensor Value: 2626" • "Sensor Value: 2633"- "Sensor Value: 2611" • "Sensor Value: 2597"- "Sensor Value: 2603" The red LED in Image 1 is glowing, confirming gas detection. Sensor values stayed between 2597-2672
Conclusion	• Gas sensor calibration works automatically • Detection threshold system is reliable • Buzzer and LED alerts activate when gas is present • Sensor readings are consistent and stable • System is suitable for safety monitoring in homes, labs, or kitchens

4.2.1 Result

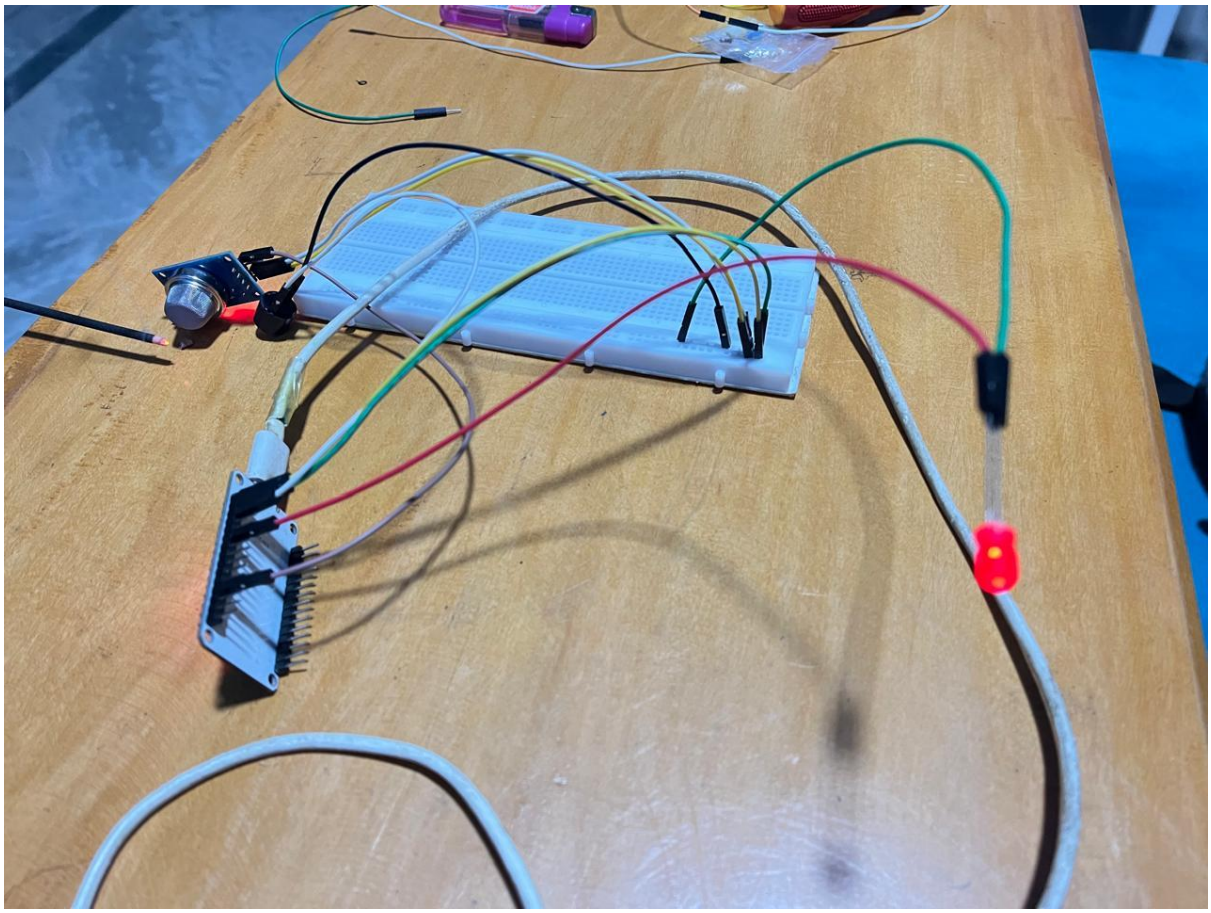

Figure 31 Gas Sensor

4.2.2 Result

Figure 32 Gas Sensor Output

4.3 Test 3: Smart Fan

Table 3 Test of Smart Fan

Test 3	**Smart Fan**
Objective	To test WiFi-based control of an electric fan through a web interface accessible from smartphones
Action	ESP32 connects to WiFi network (SSID: "ESP32", Password: "12345678") and creates a web server on port 80. Users access the web page showing "ESP32 Relay Control" with two buttons: "Turn OFF" (red) and "Turn ON" (green). Clicking these buttons sends commands to toggle relay 2, which controls the connected fan
Expected Result	• ESP32 connects to WiFi successfully • Web server starts and shows control interface • Web page displays red "Turn OFF" and green "Turn ON" buttons • Clicking buttons instantly switches the fan ON/OFF • Serial monitor shows "Relay 2 OFF" or "Relay 2 ON" • Fan responds to commands immediately
Actual Result	System worked perfectly. Serial monitor showed: • "Relay 2 OFF" • "Relay 2 ON" • "Relay 2 OFF" • "Relay 2 ON" • "Relay 2 OFF" • "Relay 2 OFF" • "Relay 2 ON" Image shows: • Phone displaying "ESP32 Relay Control" web interface • Red "Turn OFF" button and green "Turn ON" button clearly visible • Green and red LEDs glowing on ESP32 board (indicating relay active) • Clean web interface easy to use The relay controlling the fan responds instantly with no delay

Conclusion	• WiFi connection is stable and reliable • Web interface loads quickly and is user-friendly • Relay switches the fan instantly when buttons are clicked • Can control fan from anywhere within WiFi range • System is perfect for smart home fan automation • Simple two-button interface makes it accessible for all users • No physical switches needed - full remote control via smartphone

4.3.1 Result

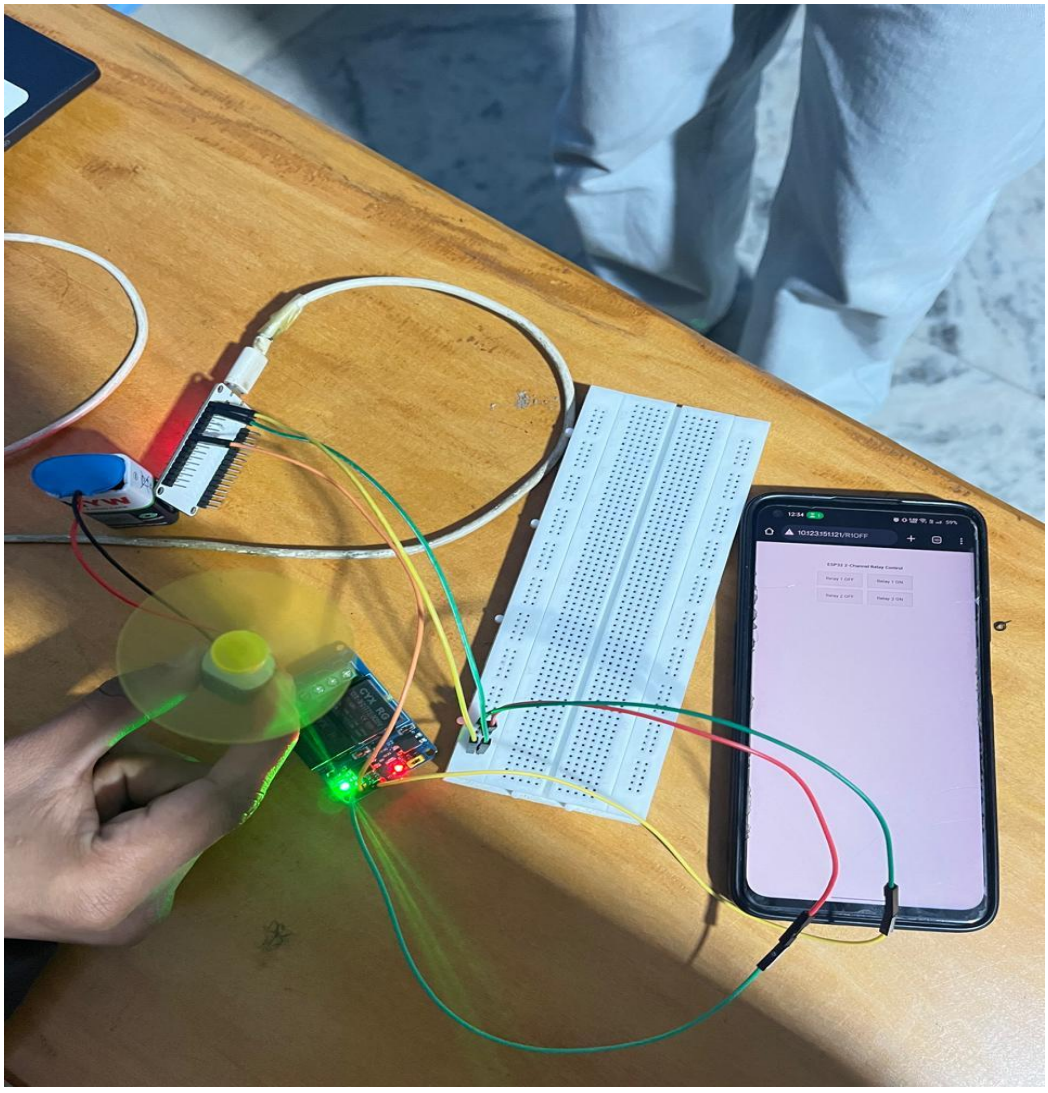

Figure 33: Smart Fan

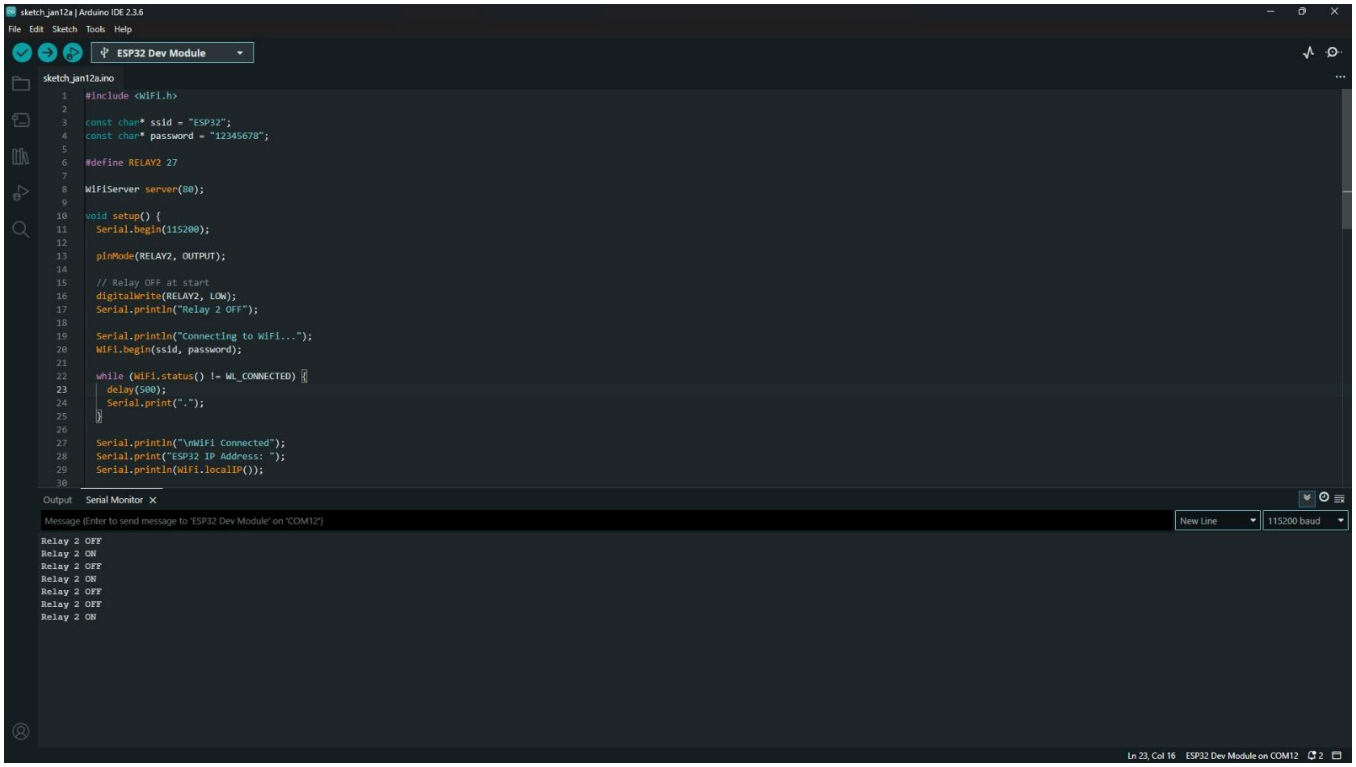

Figure 34 Smart Fan Output

4.4 Test 4: Ultra Sonic Water pump

Table 4 Test of Ultra Sonic Water pump

Test 4	**Ultra Sonic Water pump**
Objective	To test automatic water pump control using ultrasonic distance sensor (HC-SR04) that measures water level or object distance and controls relay accordingly
Action	HC-SR04 ultrasonic sensor continuously sends sound waves and measures the time they take to bounce back. This calculates distance in centimeters. When distance reaches a specific threshold, the relay (pin 27) automatically turns the water pump ON or OFF. The system also includes manual override and WiFi control through a web interface
Expected Result	• Ultrasonic sensor continuously measures distance • Distance values appear in serial monitor in centimetres • When water level changes (distance changes), relay responds • Web interface shows "Automatic Mode", "Turn ON", "Turn OFF", and "Relay State ON" buttons • System can switch between automatic and manual modes
Actual Result	System worked perfectly. Serial monitor displayed continuous distance measurements: • Distance: 4.22 cm • Distance: 3.33 cm • Distance: 4.22 cm • Distance: 4.22 cm • Distance: 3.33 cm • Distance: 3.33 cm • Distance: 3.33 cm • Distance: 3.33 cm • Distance: 3.33 cm • Distance: 3.33 cm • Distance: 5.40 cm • Distance: 4.53 cm • Distance: 4.53 cm • Distance: 4.51 cm

	• Distance: 4.51 cm • Distance: 4.53 cm Image shows phone displaying "Relay Control" web interface with all control buttons visible
Conclusion	• Ultrasonic sensor measures distance accurately (precision: ±0.3 cm) • Readings update smoothly in real-time without delays • Distance values are stable and consistent (mostly 3.33-4.53 cm range) • WiFi web interface provides both automatic and manual control options • Relay can be controlled based on distance threshold (automatic mode) • System is perfect for automatic plant watering, tank level monitoring, or smart irrigation • Combination of sensors and WiFi makes it a complete IoT solution

4.4.1 Result

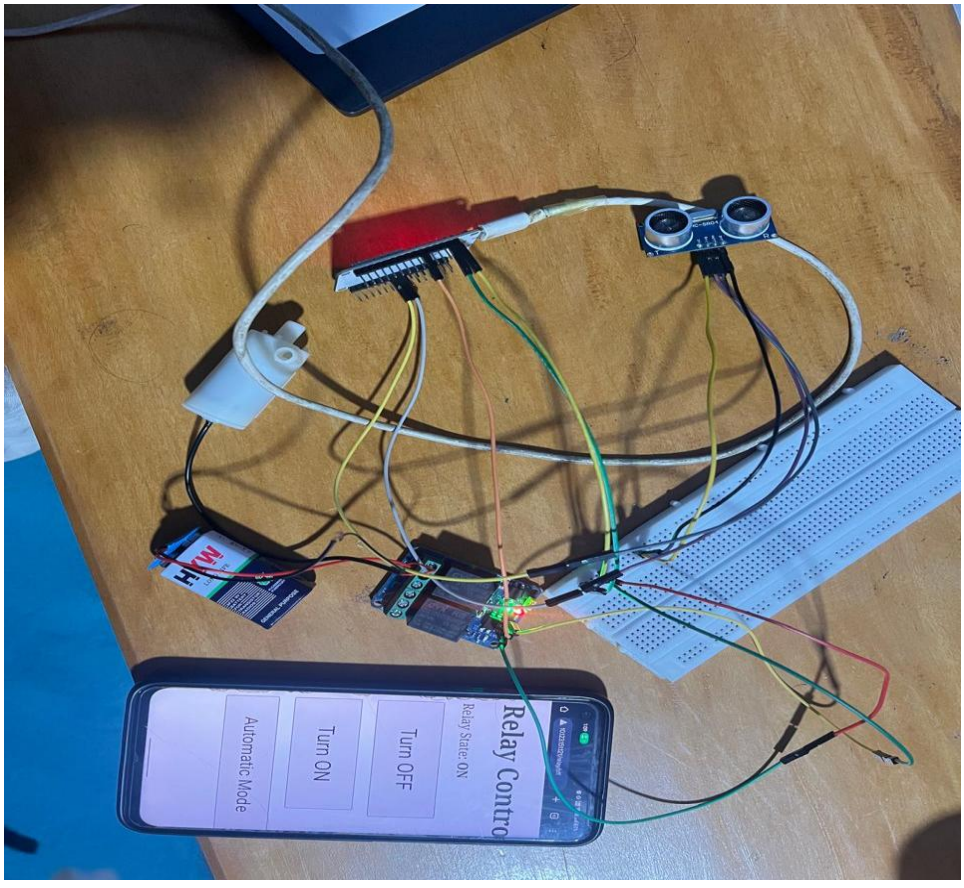

Figure 35 Ultra Sonic Water Pump

Figure 36 Ultra Sonic Water Pump Output

4.5 Test 5: Smart Bulb

Table 5 Test of Smart Bulb

Test 5	**Smart bulb**
Objective	To test WiFi-based control of a light bulb through a web interface accessible from smartphones
Action	ESP32 connects to WiFi network (SSID: "ESP32", Password: "12345678") and creates a web server on port 80. Users access the web page showing "ESP32 Relay Control" with two buttons: "Turn OFF" (red) and "Turn ON" (green). Clicking these buttons sends commands to toggle relay 2, which controls the connected light bulb
Expected Result	• ESP32 connects to WiFi successfully • Web server starts and is accessible via IP address • Web interface displays with red "Turn OFF" and green "Turn ON" buttons • Clicking buttons instantly switches the bulb ON/OFF- Serial monitor shows "Relay 2 OFF" or "Relay 2 ON" • Physical bulb lights up when turned ON and goes dark when turned OFF
Actual Result	System performed excellently. Serial monitor showed: • "Relay 2 OFF" • "Relay 2 ON" • "Relay 2 OFF" • "Relay 2 ON" • "Relay 2 OFF" • "Relay 2 OFF" • "Relay 2 ON" Image clearly shows: • Phone displaying "ESP32 Relay Control" web interface • Red "Turn OFF" button and green "Turn ON" button visible • Red LED/bulb glowing brightly on the left side (bulb is ON) • ESP32 relay module with indicator LEDs showing active state

	Bulb responds instantly to button clicks - no delay between command and light response
Conclusion	• WiFi connection is stable with no disconnections • Web interface is clean, simple, and responsive • Relay switches the bulb instantly when buttons are clicked • Physical bulb lights up correctly when relay is ON • Can control lighting from anywhere in the house via smartphone • Perfect for smart home lighting automation • No need for physical light switches - full wireless control • System can be expanded to control multiple bulbs/lights

4.5.1 Result

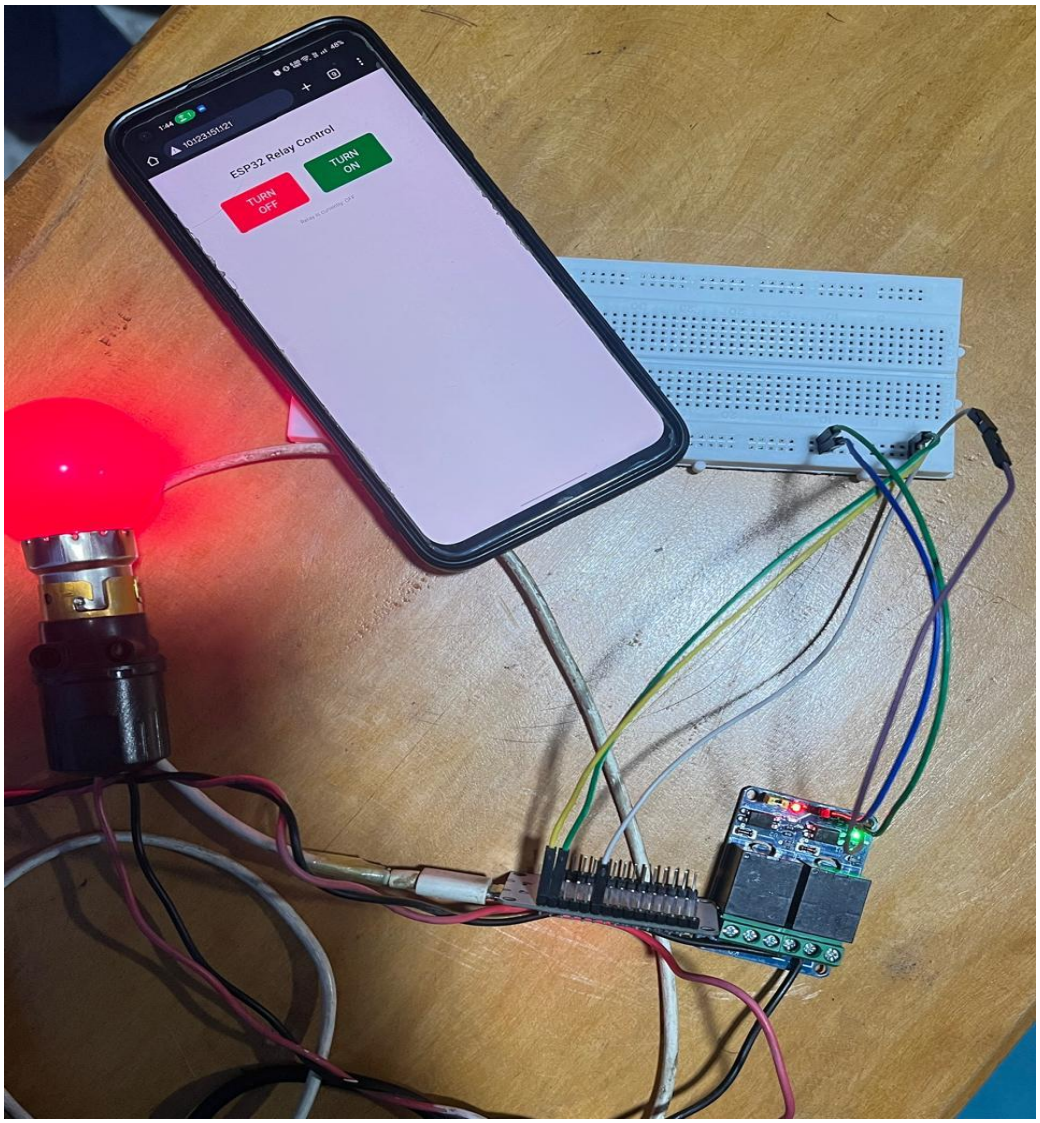

Figure 37 Smart Bulb

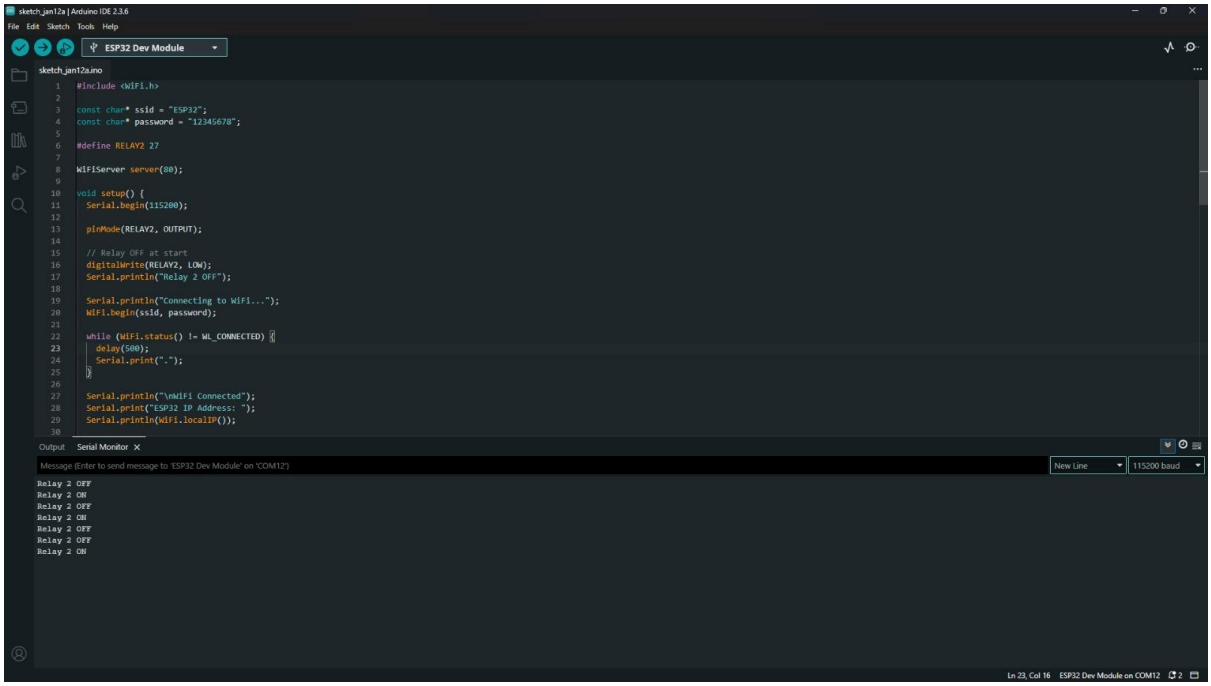

Figure 38 Smart Bulb Output

4.6 Test 6: IR Sensor

Table 6 Test of IR Sensor

Test 6	**IR Sensor**
Objective	To test if IR (Infrared) sensor can detect objects and automatically control servo motor. IR Sensor was used instead of PIR Sensor because PIR sometime gave false readings.
Action	IR sensor continuously monitors for objects. When it detects an object like someone's hand or body nearby, ESP32 signals servo motor to rotate to 180 degrees to open door. When no object is detected, then servo motor returns to closed position. Red LED provides visual feedback when object is detected
Expected Result	When object comes near IR Sensor: • Servo should rotate to open position that is 180 degrees • Red LED should light up • Serial monitor shows "Object detected" When no object: • Servo returns to closed position
Actual Result	System worked perfectly. Serial monitor showed continuous detection: • "Object Detected" • "Object Detected" Repeated 15+times Image shows Red LED glowing brightly at top when object detected and servo motor rotate to open position
Conclusion	• IR Sensor detects objects accurately and consistently • No false reading and more reliable than PIR Sensor • Servo motor responds correctly to detection • Detection is stable and shows continuous "Object Detected" message • IR Sensor is better choice than PIR for this System • System is suitable for automatic door, object detection and proximity sensing

4.6.1 Result

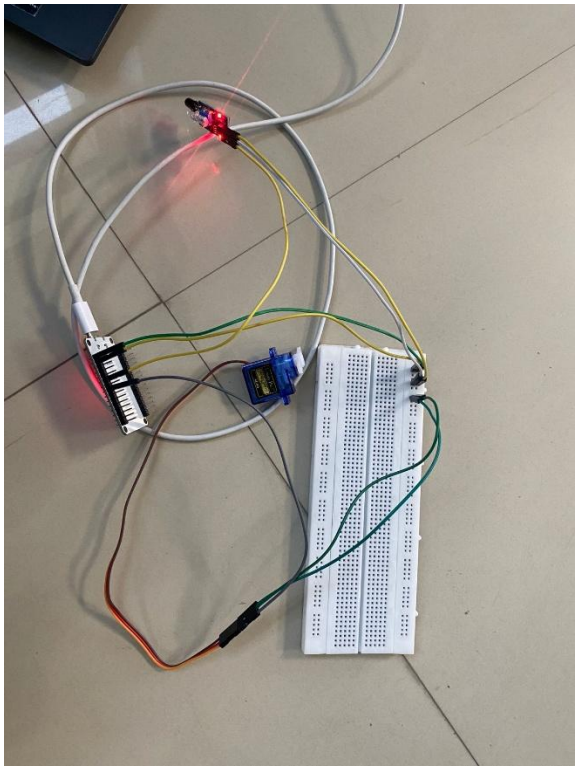

Figure 39 IR Sensor with Servo Motor

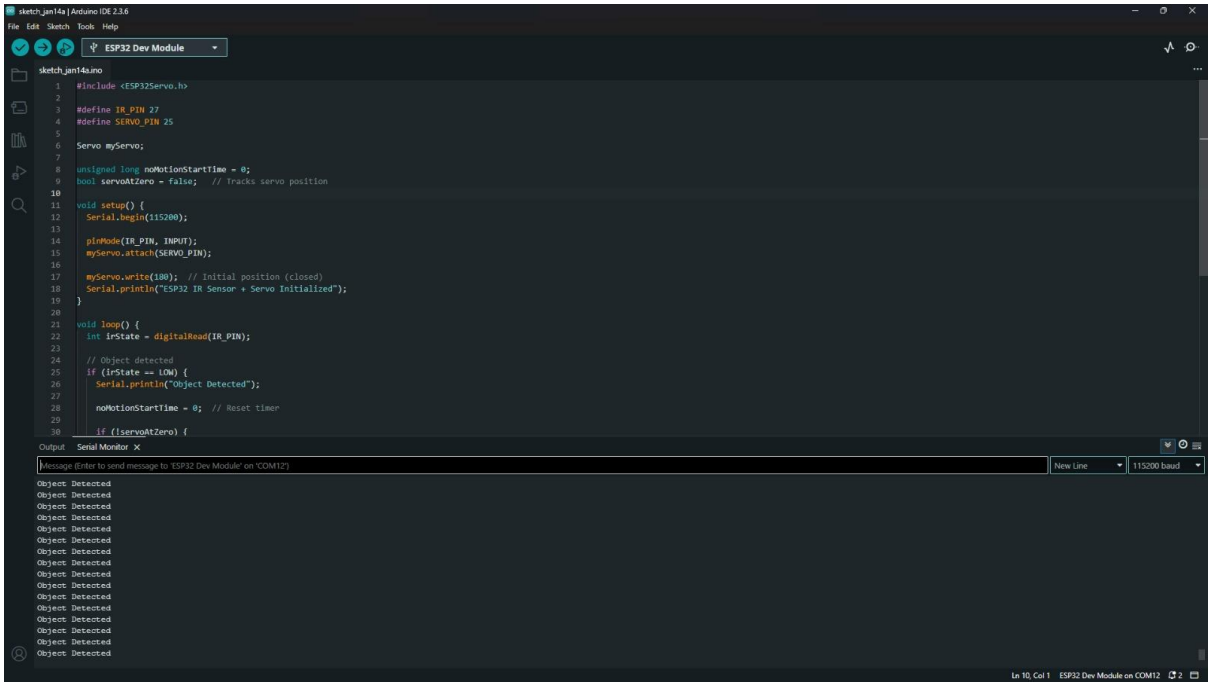

Figure 40 IR Sensor with Servo Motor Outpu

4.7 Test 7: Gas Sensor Only

Table 7 Test of Gas Sensor Only

Test 7	**Gas Sensor Only**
Objective	To verify MQ-2 Gas sensor can detect presence of gas and tigger appropriate alerts
Action	• Upload code to ESP32 • Expose sensor to gas source like lighter gas etc • Monitor serial output for detection messages
Expected Result	• "Gas sensor calibrated" message appears after setup • "Gas detected" displays when gas is present • "Gas level normal" displays when no gas is detected • LED indicator illuminates during gas detection
Actual Result	• Calibration successful • Sensor correctly detects gas presence • Serial monitor shows alternating "Gas detected" and "Gas level normal" messages • Sensor responds to gas concentration changes
Conclusion	MQ-2 Gas Sensor is functioning correctly and reliably detects gas presence above threshold level

4.7.1 Result

Figure 41: Gas Sensor only

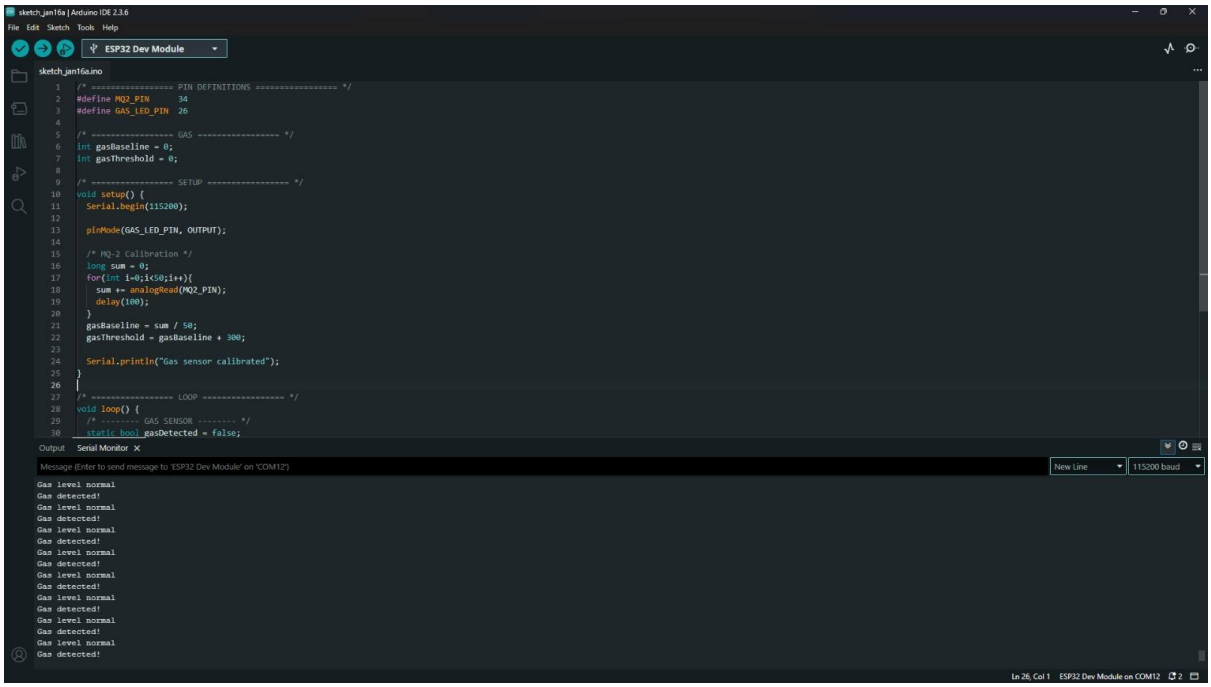

Figure 42:Gas sensor running output

4.8 Test 8: IR Sensor Only

Table 8 Test of IR Sensor Only

Test 8	**IR Sensor Only**
Objective	To verify IR motion sensor can accurately detect motion and responds appropriately when movement is present or absent
Action	• Upload code to ESP32 • Wave hand or move in front of sensor • Stay still to test "No motion" state
Expected Result	• "Motion detected" message display when movement is detected • "No motion" message displays when no movement is present • Timer starts on first motion detection • Sensor responds consistently to movement changes
Actual Result	• Sensor successfully detects motions • Serial monitors correctly display "Motion detected" when movement occurs • "No motion" display when stationary • No Motion Start timer function properly • Sensor responds reliably to presence/absence of movement
Conclusion	IR Motion Sensor is functioning correctly and reliably detects human movement within its detection range

4.8.1 Result

Figure 43:IR Sensor Only

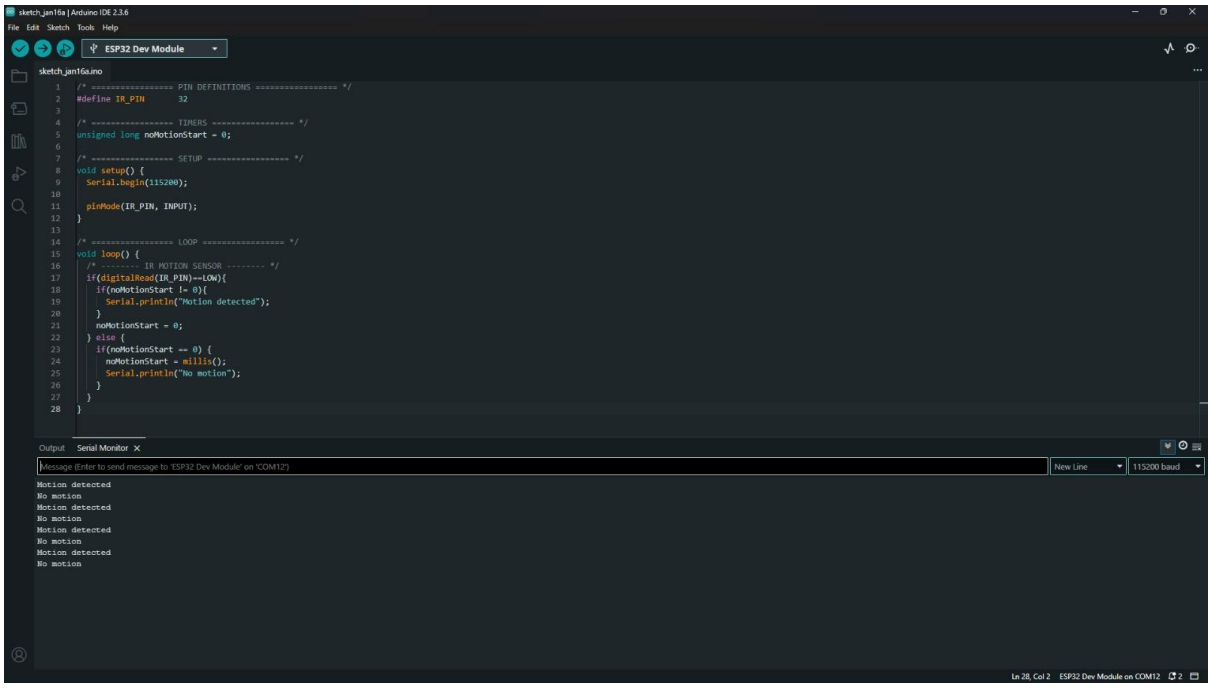

Figure 44:IR Sensor Only Output

4.9 Test 9: Servo Motor Only

Table 9 Test of Servo Motor Only

Test 9	**Servo Motor Only**
Objective	To verify servo motor can move accurately between different positions and function as an automated door mechanism
Action	• Upload servo test code to ESP32 • Connect servo motor and set initial servo position to 180 degrees (door closed) • Programmed servo to move to 0 degrees (door open) • Programmed servo to move to 180 degrees (door close) • Add 1 second delays between movements • Monitor serial output for position confirmation
Expected Result	• Servo initializes at 180 degrees • Serial monitor displays “Servo set to 180 degrees • Servo moves smoothly to 0 degrees • After delay, servo returns to 180 degrees • Serial monitor display “Servo moved to 180 degrees” • Movement repeats in continuous loop • No stuttering or jerky motion
Actual Result	Servo function correctly: • Initial position set to 180 degrees confirmed • Serial output shows: “Servo moved to 0degrees” • Serial output shows: “Servo moved to 180degrees” • Movement cycle repeating successfully • Servo transition smoothly between positions • 1 Second delays working as programmed

	• Door mechanism (model) opens and close properly
Conclusion	Servo motor is functioning correctly and reliably moves between 0 degrees and 180 degrees position. Automated door mechanism operated smoothly with proper timing delays. Servo responds accurately to commands and maintains position stability and suitable for use in automated door control systems

4.9.1 Result

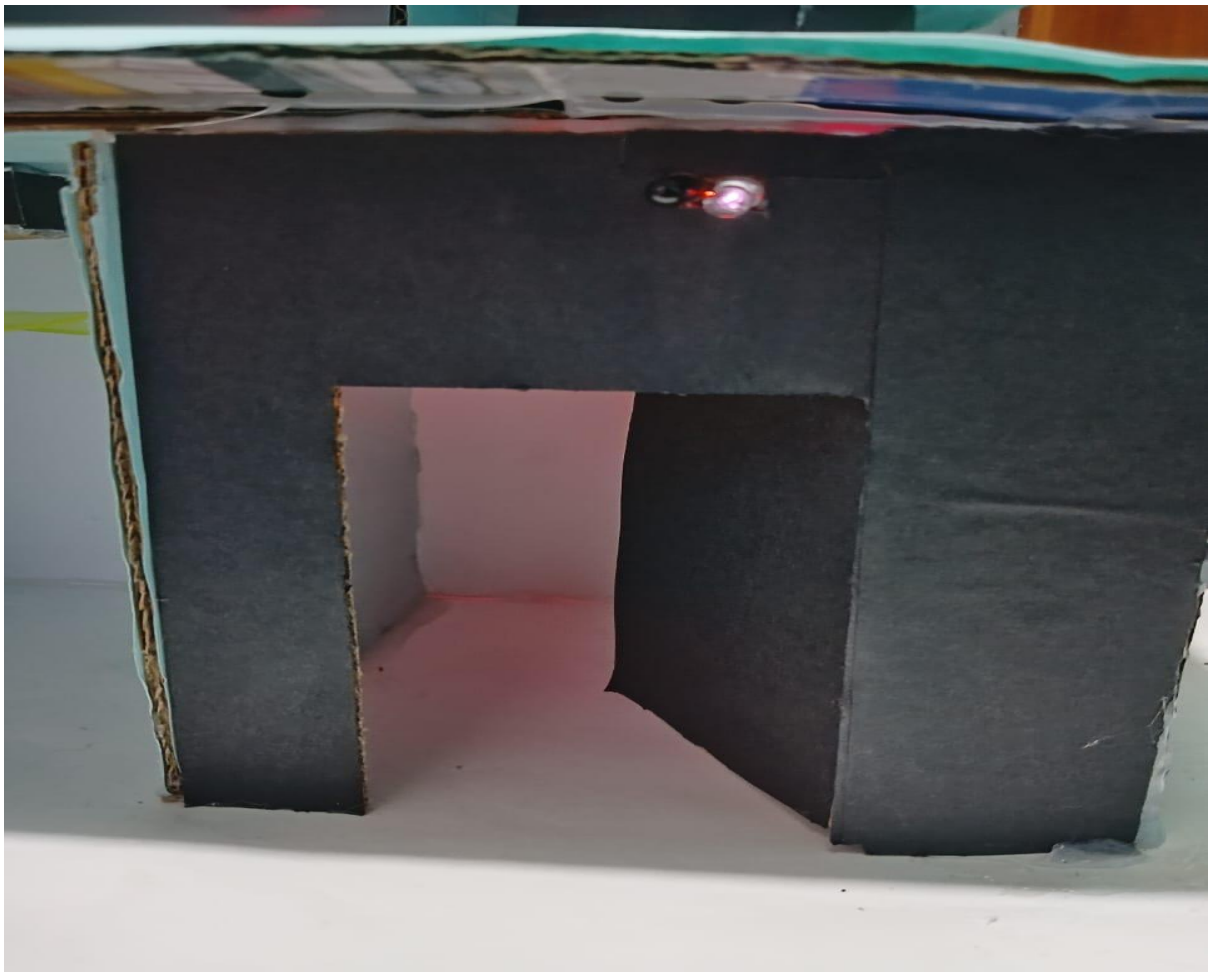

Figure 45:Servo Motor Only

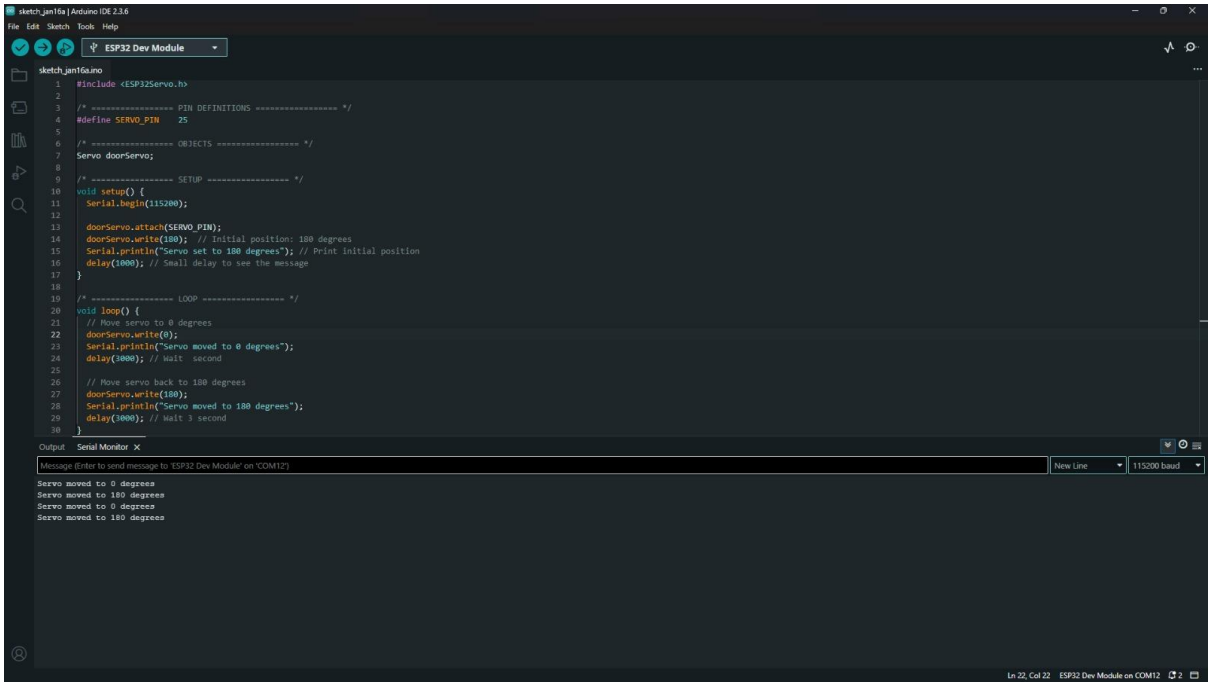

Figure 46:Servo Motor Only Output

4.10 Test 10: Overall System Testing

Table 10 Test of Overall System Testing

Test 10	**Overall system testing**
Objective	To verify all component (gas sensor, IR sensor, ultrasonic sensor, buzzer, LED indicators and waterpump) working together as an integrated smart monitoring and alert system
Action	• Attempted to upload complete integrated code to ESP32 • Connected system components including gas sensor, IR sensor, ultrasonic sensor, buzzer, LEDs and pump relay
Expected Result	• Code uploads successfully to ESP32 • Gas detection tiggers buzzer and gas LED • Ultrasonic sensor controls water pump based on distance reading • All sensors operate independently without interference • System provides serial monitor feedback for all operation
Actual Result	• Error: “Could not open COM12, the port is busy or doesnot exist” • Upload error: exit status • System could not be tested as code failed to deploy to ESP32 • Physical hardware setup is complete and ready for testing
Conclusion	Overall system testing could not be completed due to upload error.

4.10.1 Result:

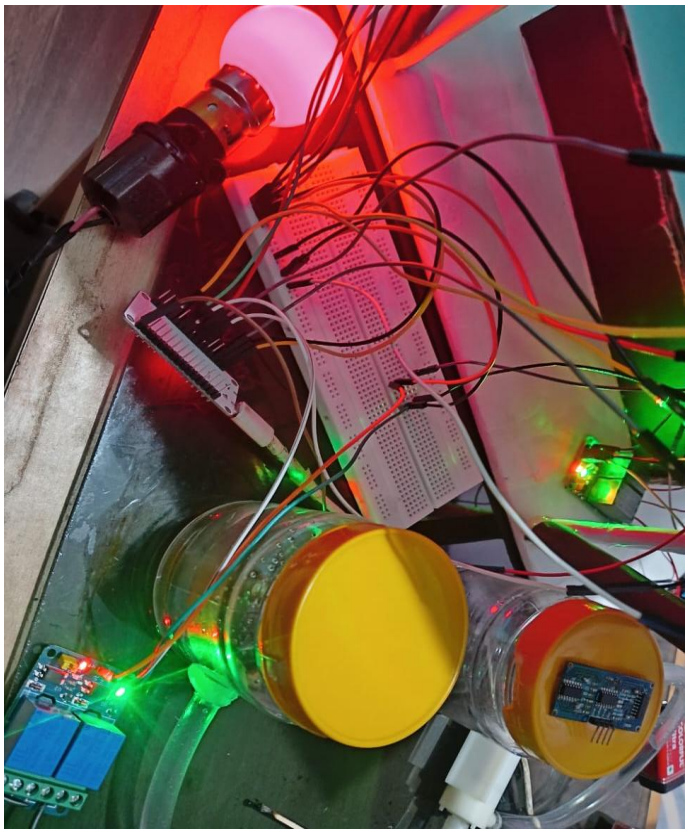

Figure 47:Overall System Testing

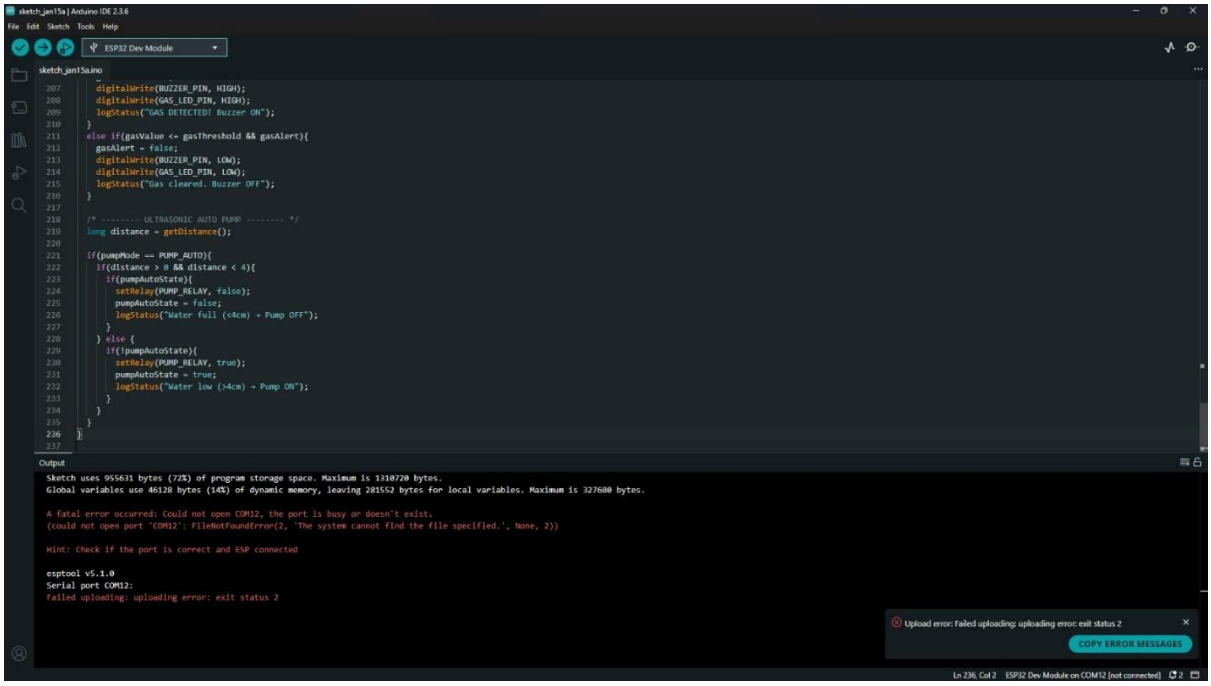

Figure 48:Overall System Testing Output

4.11 Test 11: Overall System Testing

Table 11 Test of Overall System Testing

Test 11	**Overall system testing**
Objective	To verify all component (gas sensor, IR sensor, ultrasonic sensor, buzzer, LED indicators, bulb, fan, pump) work together as an integrated smart home automation and safety system
Action	• Upload complete integrated code to ESP32 • Connected to WiFi network • Initialized all components (Bulb, fan, pump, servo motor) • Tested IR motion sensor for door control • Tested gas sensor for safety alerts • Monitored serial output for all system activities
Expected Result	• WiFi connects successfully and displays IP address • All devices initialize to OFF state (Bulb OFF, Fan OFF, Pump OFF) • IR sensor detects motion and triggers door servo to open • Door closes automatically after no motion for 2000ms • Gas sensor detects gas and triggers alert • System returns to normal when gas clears • Server handles client requests • All status changes logged to serial monitor
Actual Result	System Fully Operational: • WiFi connected successfully • Initial state: Bulb OFF, Fan OFF, Pump OFF • Motion detection working: "Motion detected" → "Door opened" • Door auto-close working: "No motion: Door closed" after timeout • Gas detection working: "Gas detected!" alerts triggered

	• Gas clearing detected: "Gas level normal" • Web server responding to client requests • All components coordinating properly without conflicts
Conclusion	Overall system integration is successful. All sensors (IR motion, MQ-2 Gas), actuators (servo door, bulb, fan, pump) and WiFi connectivity work perfectly. System demonstrates proper IoT functionality with motion-based door automation, gas safety monitoring and remote-control capabilities via web server. Real time status monitoring confirmed through serial output.

4.11.1 Result

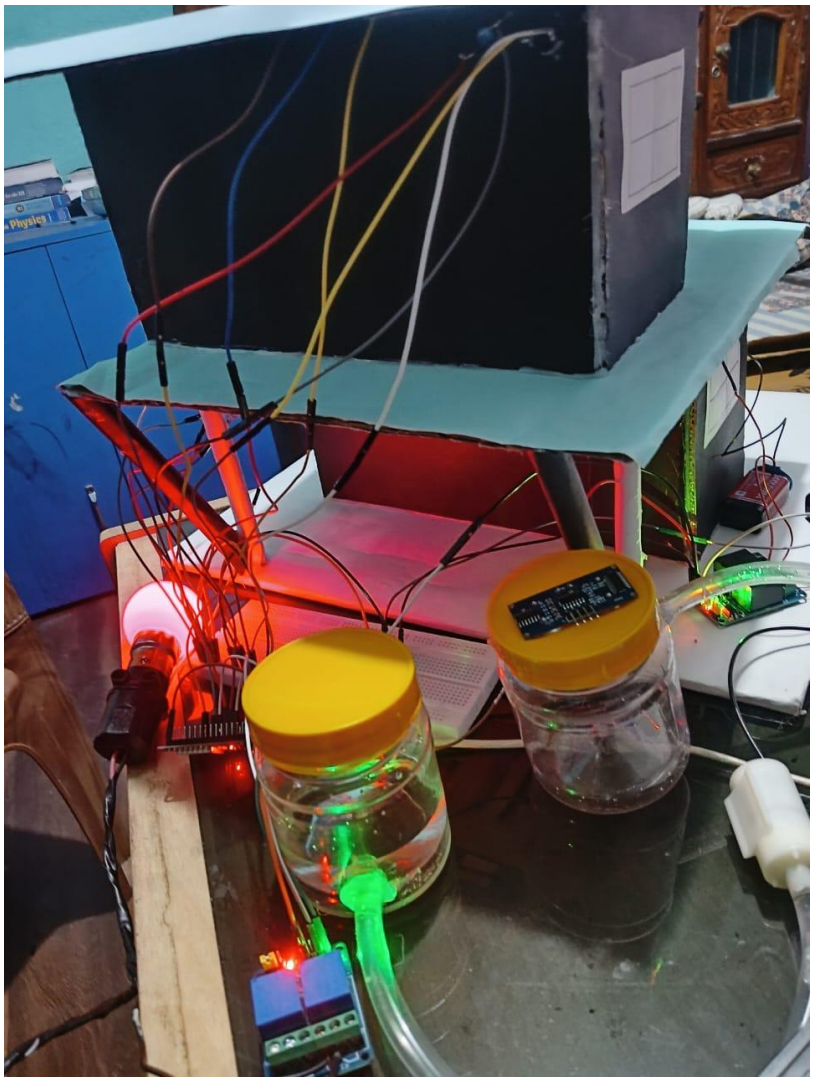

Figure 49:Overall System

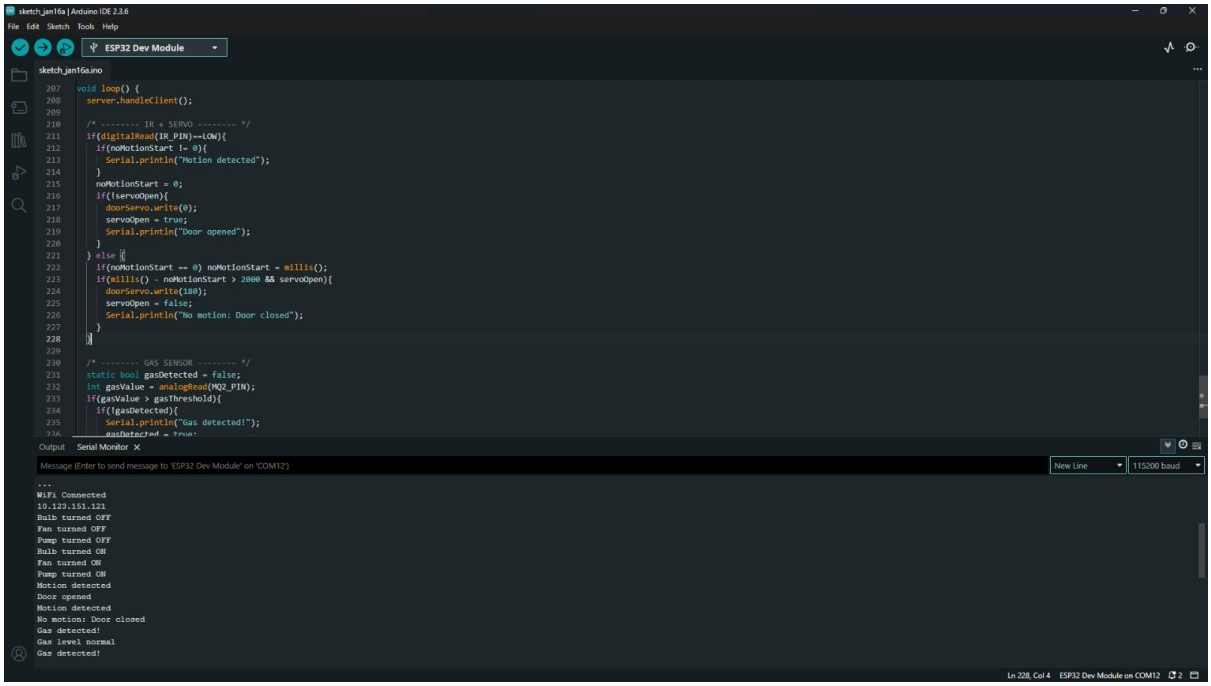

Figure 50:Overall System Testing Output

4.12 Test 12: Water pump

Table 12 Water Pump

Test 12	**Water pump**
Objective	To verify water pump can be manually controlled through the web dashboard and ESP32, functioning correctly in ON/OFF states for water circulation or transfer
Action	• Connected water pump to relay module controlled by ESP32 • Integrated pump control in web dashboard • Tested manual ON operation via web interface • Tested manual OFF operation via web interface • Verified pump motor activation when commanded ON • Verified pump motor stops when commanded OFF • Checked relay switching behaviour • Monitored serial output for pump status changes
Expected Result	• Pump turns ON when "ON" button clicked on dashboard • Pump turns OFF when "OFF" button clicked on dashboard • Relay module switches correctly (audible click) • Water pump motor runs when activated • Water flow occurs during ON state • Pump stops completely during OFF state • Dashboard displays current pump status (ON/OFF) • Serial monitor logs pump state changes • No electrical issues or overheating • Remote control responsive and reliable
Actual Result	Pump control successful: • Manual ON/OFF control working via web dashboard

	• Dashboard shows "Pump: ON" status (as visible in screenshot) • Relay module switching correctly • Water pump motor activates when ON command received • Pump stops when OFF command received • Realtime status updates displayed on dashboard • ESP32 successfully controls relay for pump operation • No integration issues with manual control • Simpler implementation than ultrasonic-automated version • Reliable and responsive user control
Conclusion	Water pump manual control is functioning correctly. The decision to remove ultrasonic sensor automation and implement manual control was successful, simplifying both hardware integration and software complexity while maintaining full functionality. User can reliably control pump remotely via web dashboard. System is stable and ready for deployment. Manual control provides more predictable operation suitable for cloud integration.

4.13 Test 13: Cloud

Table 13 Cloud

Test 12	**Cloud**
Objective	To verify smart home system can communicate with cloud services, host a web-based dashboard accessible remotely and display real time sensor data and device control via internet connectivity
Action	• Deployed Smart Home Dashboard to GitHub Pages (sarobar-001.github.io/IOT/) • Connected ESP32 to WiFi network • Established MQTT/HTTP communication between hardware and web dashboard • Accessed dashboard via web browser from remote location • Tested real-time data updates from sensors (Gas, Motion) • Tested remote control of devices (Bulb, Fan, Pump, Door Servo) • Verified bidirectional communication between cloud and hardware
Expected Result	• Web dashboard loads successfully from GitHub Pages • All device controls (Bulb, Fan, Pump) display ON/OFF toggle buttons • Motion Sensor shows current status (Motion/No Motion) • Gas Sensor displays realtime values and status (Normal/Alert) • Door Servo shows current position (Open/Closed) • User can remotely control devices via web interface • Changes on dashboard reflect on physical hardware • Hardware sensor updates appear on dashboard in realtime • Dashboard accessible from any device with internet connection
Actual Result	Cloud integration successful: • Dashboard hosted at sarobar-001.github.io/IOT/ and loading properly

	• All 6 components displayed with proper icons and labels • Control buttons visible: Bulb (ON), Fan (ON), Pump (ON) • Motion Sensor displaying: "No Motion" • Gas Sensor showing: "Normal" status with Gas Value: 1438 • Door Servo displaying: "Closed" STATUS- Professional UI with dark purple theme • Responsive card-based layout • Realtime sensor data successfully transmitted to cloud • Remote monitoring and control functional
Conclusion	Cloud integration is fully operational. The system successfully demonstrates IoT capabilities with bidirectional communication between ESP32 hardware and cloud-hosted web dashboard. Remote monitoring of all sensors (gas levels, motion detection) and remote control of all actuators (lights, fan, pump, door) working correctly. The GitHub Pages deployment provides accessible, professional interface for smart home management from anywhere with internet access. System ready for production use.

4.13.1 Result

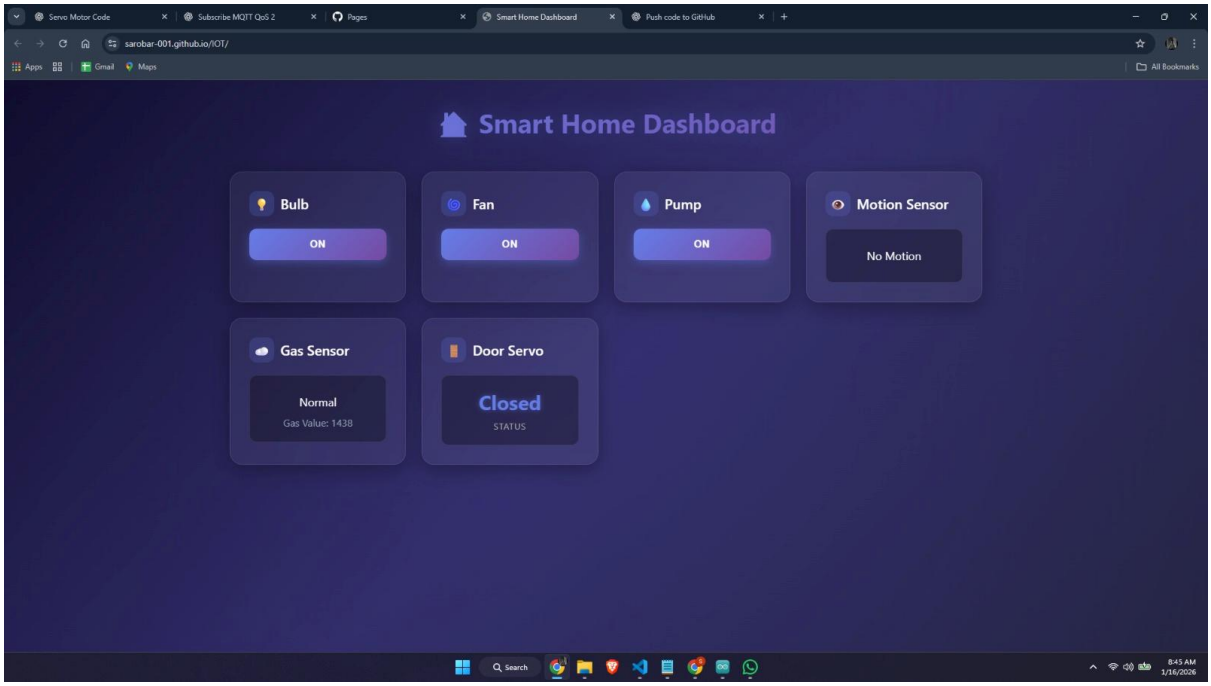

Figure 51:Cloud integration web control

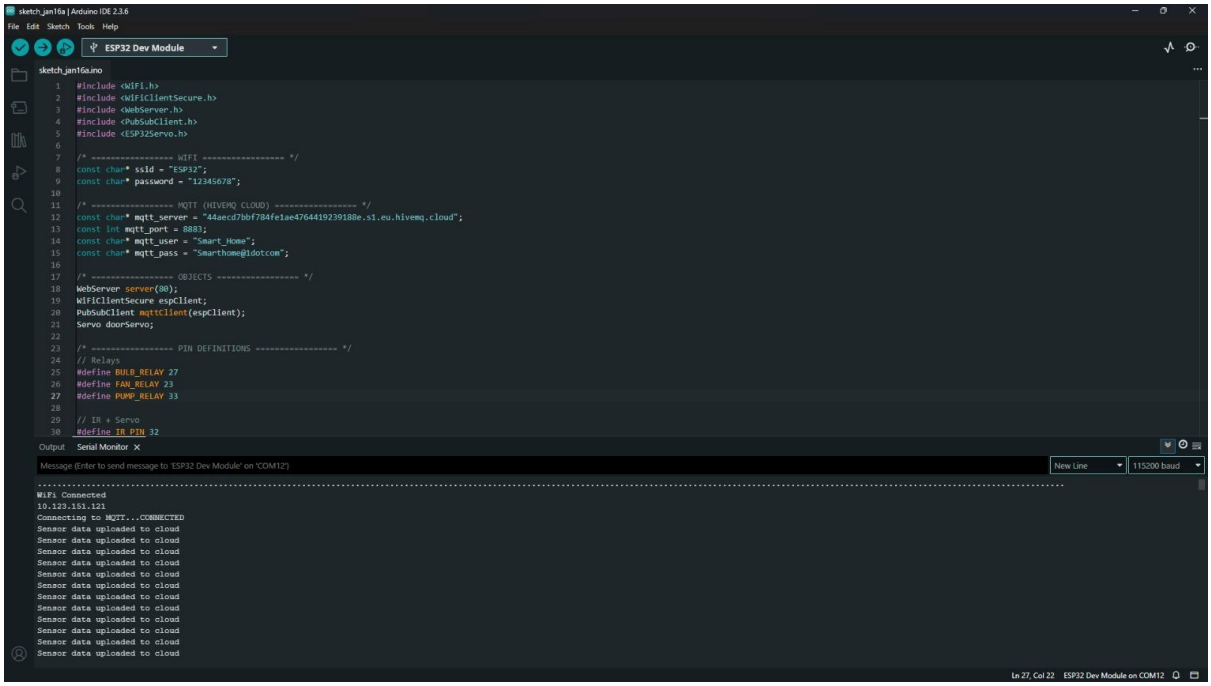

Figure 52:Output of cloud integration

5 Future Works

- **Mobile Application development:** A dedicated iOS or Android application with a graphical user interface can be developed to replace basic Bluetooth control apps, improving useability and system interaction.
- **Energy Consumption Monitoring:** Future versions of the system can include current sensors to monitor real time energy consumption and provide insight for energy optimization.
- **Enhanced Security Features:** Additional sensors such as cameras or biometric authentication can be implemented to improve home security.
- **Scalability and Multi Room Support:** System can be expanded to support multiple rooms and additional device through modular design and improved communication protocols.

  ReadMore

6 Conclusion

Through this project, the Internet of Things (IoT) principles were effectively utilized by designing and installing a smart home automation system to solve the usual challenges faced in the household including wasting energy, safety concerns, and inconvenience on manual control. The project combined sensors, actuators, wireless communication, and embedded control, and proved that the affordable and reliable automation solution can be created with the help of low-cost hardware and open-source technologies.

6.1 Teamwork and Collaboration Learning outcomes.

The project gave me a lot of learning opportunities that were well underpinned by teamwork. Team members shared other duties like assembly of hardware, embedded programming, testing and documentation of the system by working together. Such a teamwork method allowed the team to find the limitations of the system including the false motion detection due to the PIR sensors and find a better solution together, in this case, IR-based detection. In the process, the team did not only acquire technical skills but also the necessary professional competencies such as the ability to communicate, solve problems, manage time and make decisions together as a team, which are vital in actual engineering setting.

6.2 ESP Programming with Hardware Components.

One of the fundamental technical contributions to the project involved the ability to seamlessly combine ESP32-based programming and several hardware devices. Firmware was programmed with the Arduino IDE to process sensor readings, process decision logic, and actuators, such as relays, servo motors, buzzers, and LEDs, in real time. The project increased the knowledge on GPIO control, PWM signals, analogy to digital conversion, sensor calibration etc. The integration of hardware and software was done seamlessly, and this showed efficient embedded system design, high levels of reliability during automation and safety-related responses.

6.3 Wi-Fi, Clouds and Software.

To allow control of appliances in a web interface hosted on the ESP32, the system added Wi-Fi connectivity. This brought in feasible ideas of remote access, network communication, and scaled system design. Even though this project could not go all the way to full cloud integration, the background information pertaining to cloud-based monitoring and future data analytics was laid. Software platforms like Arduino IDE and TinkerCad facilitated effective

development, emulation and debugging, strengthening best practices in IoT software engineering.

6.4 Final Reflection

To sum up, the project did not fail to meet its goals as it provided a functional, secure, and user-friendly smart home automation prototype. The project resonated well with the learning outcomes of the Cloud Computing and Internet of Things module by integrating the learning concepts of teamwork-driven learning, embedded ESP programming, integrated hardware components, and network-based control. Although the system has weaknesses, including the short operating range and the incomplete application of cloud, it has a good ground upon which further improvements can be made. All in all, the project was very strong in technical knowledge and teamwork abilities to develop more advanced systems based on IoT and cloud computing.

7 References

Ashleyvu. (2023) *Medium* [Online]. Available from: https://medium.com/@ashleyvu239/iot-project-water-pumping-management-system-d8648051c2c9 [Accessed 25 November 2025].

Expert, A. (2024) *Water Tank and Water Pump Automation* [Online]. Available from: https://arduinoexpert.com/water-tank-and-water-pump-automation-using-by-esp32/ [Accessed 27 November 2025].

Jain, R. (2023) *Automatic LPG Leakage Detection Using Arduino* [Online]. Available from: https://www.electronicsforu.com/electronics-projects/lpg-gas-leakage-detection [Accessed 24 November 2025].

Pro, I.D. (2023) *DIY Voice Controlled Home Automation with Arduino and Bluetooth* [Online]. Available from: https://iotdesignpro.com/articles/diy-voice-controlled-home-automation-with-arduino-and-bluetooth [Accessed 20 November 2025].

Saddam. (2015) *Automatic Door Opener* [Online]. Available from: https://circuitdigest.com/microcontroller-projects/automatic-door-opener-project-using-arduino [Accessed 26 November 2025].

8 Appendix

8.1 Appendix A: Source Code

8.1.1 IR Sensor

```
#include <ESP32Servo.h>

#define IR_PIN 27
#define SERVO_PIN 25

Servo myServo;

unsigned long noMotionStartTime = 0;
bool servoAtZero = false;   // Tracks servo position

void setup() {
  Serial.begin(115200);

  pinMode(IR_PIN, INPUT);
  myServo.attach(SERVO_PIN);

  myServo.write(180);  // Initial position (closed)
  Serial.println("ESP32 IR Sensor + Servo Initialized");
}

void loop() {
  int irState = digitalRead(IR_PIN);
```

```
  // Object detected
  if (irState == LOW) {
    Serial.println("Object Detected");

    noMotionStartTime = 0;  // Reset timer

    if (!servoAtZero) {
      myServo.write(0);
      servoAtZero = true;
      Serial.println("Servo moved to 0°");
    }
  }
  // No object detected
  else {
    if (noMotionStartTime == 0) {
      noMotionStartTime = millis();  // Start timer
      Serial.println("No object detected, timer started");
    }

    // Check if 2 seconds have passed
    if (millis() - noMotionStartTime >= 2000) {
      if (servoAtZero) {
        myServo.write(180);
        servoAtZero = false;
        Serial.println("2 seconds passed → Servo moved to 180°");
      }
```

```
  }
 }

 delay(50); // Small delay for stability
```

8.1.2 Gas Detection

```
#define MQ2_PIN      34   // MQ-2 analog output
#define BUZZER_PIN   13   // Buzzer
#define LED_PIN      27   // Status LED

int baseline = 0;
int threshold = 0;

void setup() {
 Serial.begin(115200);

 pinMode(BUZZER_PIN, OUTPUT);
 pinMode(LED_PIN, OUTPUT);

 digitalWrite(BUZZER_PIN, LOW);
 digitalWrite(LED_PIN, LOW);

 // ---------- MQ-2 CALIBRATION ----------
 Serial.println("Calibrating... Keep sensor in clean air");
 long sum = 0;
```

```
  for (int i = 0; i < 50; i++) {
    sum += analogRead(MQ2_PIN);
    delay(100);
  }

  baseline = sum / 50;
  threshold = baseline + 300;

  Serial.print("Baseline: ");
  Serial.println(baseline);
  Serial.print("Threshold: ");
  Serial.println(threshold);
  Serial.println("Calibration complete!\n");
}

void loop() {
  int sensorValue = analogRead(MQ2_PIN);
  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);

  if (sensorValue > threshold) {
    // ---------- GAS DETECTED ----------
    Serial.println("⚠ Gas detected!");

    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(LED_PIN, HIGH);
```

```
    delay(100);                // Fast blink
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(LED_PIN, LOW);
    delay(100);
  } else {
    // ---------- NO GAS ----------
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(LED_PIN, LOW);
    delay(200);
```

8.1.3 Smart Fan

```
#include <WiFi.h>

const char* ssid = "ESP32";
const char* password = "12345678";

#define RELAY2 27

WiFiServer server(80);

void setup() {
  Serial.begin(115200);

  pinMode(RELAY2, OUTPUT);
```

```
  // Relay OFF at start
  digitalWrite(RELAY2, LOW);
  Serial.println("Relay 2 OFF");

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected");
  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

  String request = "";
  while (client.connected()) {
    if (client.available()) {
```

```
char c = client.read();
request += c;

if (c == '\n') {

 // -------- Relay 2 Control --------
 if (request.indexOf("GET /R2ON") >= 0) {
  digitalWrite(RELAY2, HIGH);
  Serial.println("Relay 2 ON");
 }

 if (request.indexOf("GET /R2OFF") >= 0) {
  digitalWrite(RELAY2, LOW);
  Serial.println("Relay 2 OFF");
 }

 // -------- Web Page --------
 client.println("HTTP/1.1 200 OK");
 client.println("Content-Type: text/html");
 client.println();
 client.println("<!DOCTYPE html>");
 client.println("<html>");
 client.println("<head>");
 client.println("<style>");
 client.println("body { font-family: Arial; text-align: center; margin-top: 100px; }");
 client.println("button { width: 200px; height: 80px; font-size: 24px; margin: 10px; }");
```

```
        client.println("</style>");
        client.println("</head>");
        client.println("<body>");

        client.println("<h2>ESP32 Relay 2 Control</h2>");

        client.println("<a href='/R2ON'><button>Relay 2 ON</button></a>");
        client.println("<a href='/R2OFF'><button>Relay 2 OFF</button></a>");

        client.println("</body>");
        client.println("</html>");
        client.println();
        break;
      }
    }
  }
  client.stop();
}
```

8.1.4 Ultra Sonic Water pump

```
#include <WiFi.h>

// Wi-Fi credentials
const char* ssid = "ESP32";
const char* password = "12345678";
```

```
// Ultrasonic sensor pins
#define TRIG_PIN 5
#define ECHO_PIN 18

// Relay pin
#define RELAY2 27

// Variables
long duration;
float distanceCm;
bool manualOverride = false;  // true if manual control is active
bool relayState = false;      // current state of relay

WiFiServer server(80);

void setup() {
  Serial.begin(115200);

  // Pin setup
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RELAY2, OUTPUT);
  digitalWrite(RELAY2, LOW); // Relay OFF initially

  // Connect to Wi-Fi
  WiFi.begin(ssid, password);
```

```
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected! IP: ");
  Serial.println(WiFi.localIP());

  // Start web server
  server.begin();
}

void loop() {
  // Check for client
  WiFiClient client = server.available();
  if (client) {
    Serial.println("New Client Connected");
    String request = client.readStringUntil('\r');
    Serial.println(request);
    client.flush();

    // Handle web request for relay control
    if (request.indexOf("/relay/on") != -1) {
      manualOverride = true;
      relayState = true;
```

```
    } else if (request.indexOf("/relay/off") != -1) {
      manualOverride = true;
      relayState = false;
    } else if (request.indexOf("/auto") != -1) {
      manualOverride = false; // return to automatic mode
    }

    // HTML page with buttons
String html = "<!DOCTYPE html><html><head><title>ESP32 Relay
Control</title></head><body>";
html += "<h1>Relay Control</h1>";
html += "<p>Relay State: <strong>" + String(relayState ? "OFF" : "ON") + "</strong></p>";

// Larger buttons using inline CSS
html += "<p><a href=\"/relay/on\"><button style=\"font-size:24px; padding:20px
40px;\">Turn ON</button></a></p>";
html += "<p><a href=\"/relay/off\"><button style=\"font-size:24px; padding:20px
40px;\">Turn OFF</button></a></p>";
html += "<p><a href=\"/auto\"><button style=\"font-size:20px; padding:15px
30px;\">Automatic Mode</button></a></p>";

html += "</body></html>";


    client.println("HTTP/1.1 200 OK");
    client.println("Content-type:text/html");
    client.println();
```

```
  client.println(html);
  client.stop();
 }

 // Read ultrasonic sensor
 digitalWrite(TRIG_PIN, LOW);
 delayMicroseconds(2);
 digitalWrite(TRIG_PIN, HIGH);
 delayMicroseconds(10);
 digitalWrite(TRIG_PIN, LOW);

 duration = pulseIn(ECHO_PIN, HIGH);
 distanceCm = (duration * 0.0343) / 2;

 Serial.print("Distance: ");
 Serial.println(distanceCm);

 // Automatic relay logic if not overridden
 if (!manualOverride) {
  if (distanceCm < 7.0) {
   relayState = true;
  } else {
   relayState = false;
  }
 }
```

```
  // Apply relay state
  digitalWrite(RELAY2, relayState ? HIGH : LOW);

  delay(200);
}
```

8.1.5 Smart Bulb

```
#include <WiFi.h>

const char* ssid = "ESP32";
const char* password = "12345678";

#define RELAY2 27

WiFiServer server(80);

void setup() {
  Serial.begin(115200);

  pinMode(RELAY2, OUTPUT);

  // Relay OFF at start
  digitalWrite(RELAY2, LOW);
  Serial.println("Relay 2 OFF");
```

```
  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected");
  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

  String request = "";
  while (client.connected()) {
    if (client.available()) {
      char c = client.read();
      request += c;

      if (c == '\n') {
```

```
// -------- Relay 2 Control --------
if (request.indexOf("GET /R2ON") >= 0) {
  digitalWrite(RELAY2, HIGH);
  Serial.println("Relay 2 ON");
}

if (request.indexOf("GET /R2OFF") >= 0) {
  digitalWrite(RELAY2, LOW);
  Serial.println("Relay 2 OFF");
}

// -------- Web Page --------
client.println("HTTP/1.1 200 OK");
client.println("Content-Type: text/html");
client.println();
client.println("<!DOCTYPE html>");
client.println("<html>");
client.println("<head>");
client.println("<style>");
client.println("body { font-family: Arial; text-align: center; margin-top: 100px; }");
client.println("button { width: 200px; height: 80px; font-size: 24px; margin: 10px; }");
client.println("</style>");
client.println("</head>");
client.println("<body>");
```

```
        client.println("<h2>ESP32 Relay 2 Control</h2>");

        client.println("<a href='/R2ON'><button>Relay 2 ON</button></a>");
        client.println("<a href='/R2OFF'><button>Relay 2 OFF</button></a>");

        client.println("</body>");
        client.println("</html>");
        client.println();
        break;
      }
    }
  }
  client.stop();
}
```

8.1.6 Final Combined Code

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <ESP32Servo.h>

/* ================== WIFI ================== */
const char* ssid = "ESP32";
const char* password = "12345678";

/* ================== MQTT (HIVEMQ CLOUD) ================== */
```

```
const char* mqtt_server = "44aecd7bbf784fe1ae4764419239188e.s1.eu.hivemq.cloud";
const int mqtt_port = 8883;
const char* mqtt_user = "Smart_Home";
const char* mqtt_pass = "Smarthome@1dotcom";

/* ================= OBJECTS ================= */
WebServer server(80);
WiFiClientSecure espClient;
PubSubClient mqttClient(espClient);
Servo doorServo;

/* ================= PIN DEFINITIONS ================= */
// Relays
#define BULB_RELAY 27
#define FAN_RELAY 23
#define PUMP_RELAY 33

// IR + Servo
#define IR_PIN 32
#define SERVO_PIN 25

// Gas Sensor
#define MQ2_PIN 34
#define BUZZER_PIN 13
#define GAS_LED_PIN 26

/* ================= STATES ================= */
bool bulbState = false;
bool fanState = false;
bool pumpState = false;
```

```
bool servoOpen = false;

/* ================== TIMERS ================== */
unsigned long noMotionStart = 0;
unsigned long lastPublish = 0;

/* ================== GAS ================== */
int gasBaseline = 0;
int gasThreshold = 0;

/* ================== HELPER ================== */
void setRelay(int pin, bool state) {
  digitalWrite(pin, state ? HIGH : LOW);
}

/* ================== MQTT CALLBACK ================== */
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String msg;
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];

  if (String(topic) == "home/bulb") {
    bulbState = (msg == "ON");
    setRelay(BULB_RELAY, bulbState);
  } else if (String(topic) == "home/fan") {
    fanState = (msg == "ON");
    setRelay(FAN_RELAY, fanState);
  } else if (String(topic) == "home/pump") {
    pumpState = (msg == "ON");
    setRelay(PUMP_RELAY, pumpState);
  }
```

```
}

/* ================== MQTT CONNECT ================== */
void connectMQTT() {
  while (!mqttClient.connected()) {
    Serial.print("Connecting to MQTT...");
    if (mqttClient.connect("ESP32_SmartHome", mqtt_user, mqtt_pass)) {
      Serial.println("CONNECTED");
      mqttClient.subscribe("home/bulb");
      mqttClient.subscribe("home/fan");
      mqttClient.subscribe("home/pump");
    } else {
      Serial.print("FAILED, rc=");
      Serial.println(mqttClient.state());
      delay(2000);
    }
  }
}

/* ================== WEB UI ================== */
void handleRoot() {
  String page =
    "<!DOCTYPE html><html><body>"
    "<h1>ESP32 Smart Home</h1>"
    "<a href='/bulbOn'>Bulb ON</a><br>"
    "<a href='/bulbOff'>Bulb OFF</a><br>"
    "<a href='/fanOn'>Fan ON</a><br>"
    "<a href='/fanOff'>Fan OFF</a><br>"
    "<a href='/pumpOn'>Pump ON</a><br>"
    "<a href='/pumpOff'>Pump OFF</a><br>"
```

```
    "</body></html>";
  server.send(200, "text/html", page);
}

/* ================== WEB ROUTES ================== */
void bulbOn() {
  bulbState = true;
  setRelay(BULB_RELAY, true);
  server.sendHeader("Location", "/");
  server.send(303);
}
void bulbOff() {
  bulbState = false;
  setRelay(BULB_RELAY, false);
  server.sendHeader("Location", "/");
  server.send(303);
}
void fanOn() {
  fanState = true;
  setRelay(FAN_RELAY, true);
  server.sendHeader("Location", "/");
  server.send(303);
}
void fanOff() {
  fanState = false;
  setRelay(FAN_RELAY, false);
  server.sendHeader("Location", "/");
  server.send(303);
}
void pumpOn() {
```

```
  pumpState = true;
  setRelay(PUMP_RELAY, true);
  server.sendHeader("Location", "/");
  server.send(303);
}
void pumpOff() {
  pumpState = false;
  setRelay(PUMP_RELAY, false);
  server.sendHeader("Location", "/");
  server.send(303);
}

/* ================= SETUP ================= */
void setup() {
  Serial.begin(115200);

  pinMode(BULB_RELAY, OUTPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(PUMP_RELAY, OUTPUT);
  pinMode(IR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(GAS_LED_PIN, OUTPUT);

  setRelay(BULB_RELAY, false);
  setRelay(FAN_RELAY, false);
  setRelay(PUMP_RELAY, false);

  doorServo.attach(SERVO_PIN);
  doorServo.write(180);
```

```
  // MQ-2 calibration
  long sum = 0;
  for (int i = 0; i < 50; i++) {
    sum += analogRead(MQ2_PIN);
    delay(100);
  }
  gasBaseline = sum / 50;
  gasThreshold = gasBaseline + 300;

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected");
  Serial.println(WiFi.localIP());

  espClient.setInsecure();
  mqttClient.setServer(mqtt_server, mqtt_port);
  mqttClient.setCallback(mqttCallback);

  server.on("/", handleRoot);
  server.on("/bulbOn", bulbOn);
  server.on("/bulbOff", bulbOff);
  server.on("/fanOn", fanOn);
  server.on("/fanOff", fanOff);
  server.on("/pumpOn", pumpOn);
  server.on("/pumpOff", pumpOff);
  server.begin();
}
```

```
/* ================= LOOP ================= */
void loop() {
  server.handleClient();

  if (!mqttClient.connected()) connectMQTT();
  mqttClient.loop();

  bool motionDetected = digitalRead(IR_PIN) == LOW;
  int gasValue = analogRead(MQ2_PIN);
  bool gasDetected = gasValue > gasThreshold;

  // --- Servo Logic ---
  if(motionDetected){
    noMotionStart = 0;
    if(!servoOpen){
      doorServo.write(0);
      servoOpen=true;
    }
  } else {
    if(noMotionStart==0) noMotionStart=millis();
    if(millis()-noMotionStart>2000 && servoOpen){
      doorServo.write(180);
      servoOpen=false;
    }
  }

  // --- Gas Sensor LED/Buzzer ---
  digitalWrite(BUZZER_PIN, gasDetected ? HIGH : LOW);
  digitalWrite(GAS_LED_PIN, gasDetected ? HIGH : LOW);
```

```
  // --- Publish to Cloud every 3s ---
  if(millis()-lastPublish>=3000){
    lastPublish=millis();

    mqttClient.publish("home/bulb/state", bulbState ? "ON" : "OFF", true);
    mqttClient.publish("home/fan/state", fanState ? "ON" : "OFF", true);
    mqttClient.publish("home/pump/state", pumpState ? "ON" : "OFF", true);

    mqttClient.publish("home/motion", motionDetected ? "DETECTED" : "NONE", true);
    mqttClient.publish("home/gas/value", String(gasValue).c_str(), true);
    mqttClient.publish("home/gas/status", gasDetected ? "DETECTED" : "NORMAL", true);
    mqttClient.publish("home/servo", servoOpen ? "OPEN" : "CLOSE", true);

    Serial.println("Sensor data uploaded to cloud");
  }
}
```

8.2 Appendix B: Screenshots of System

8.2.1 Finished product (Arduino IDE)

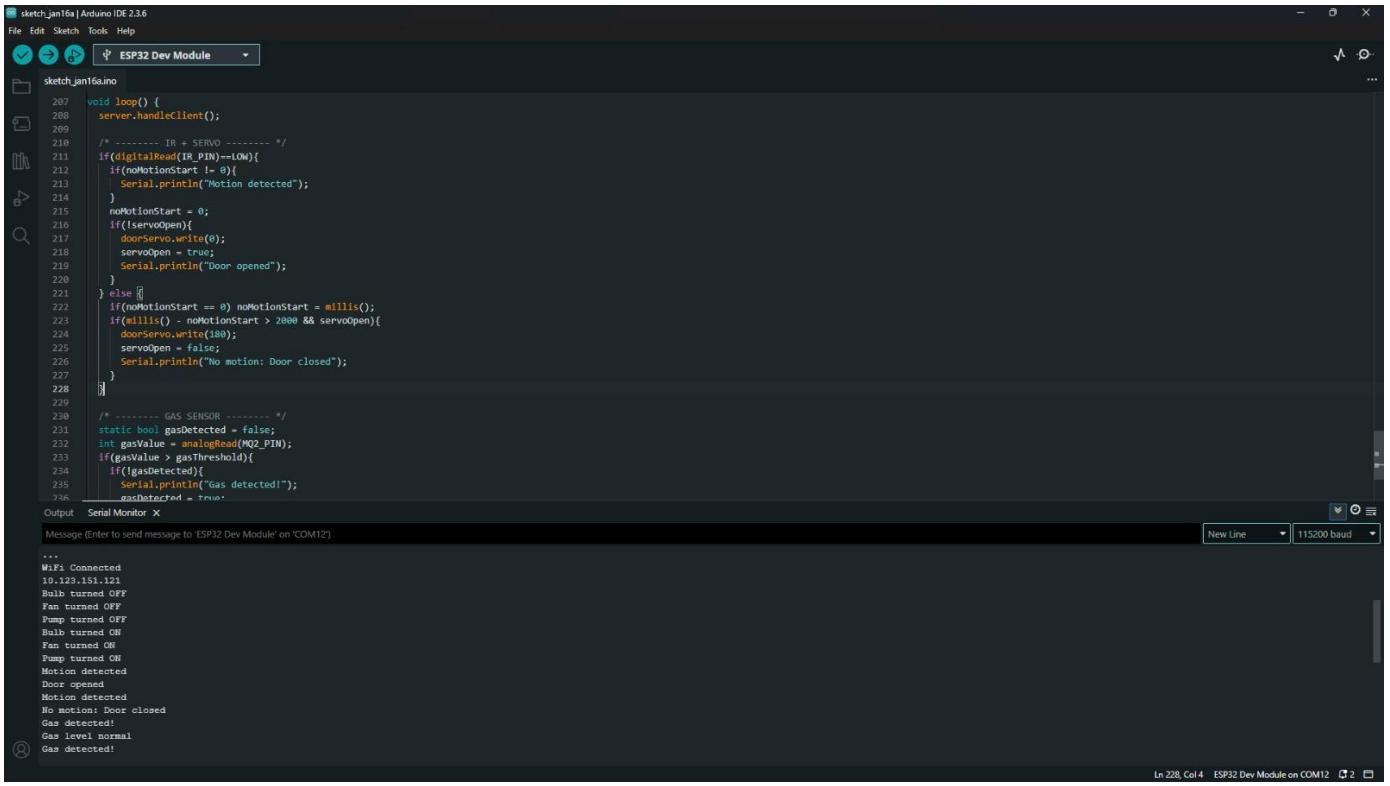

Figure 53:Finished product

8.2.2 Finished Product

Figure 54: Finished product from front POV

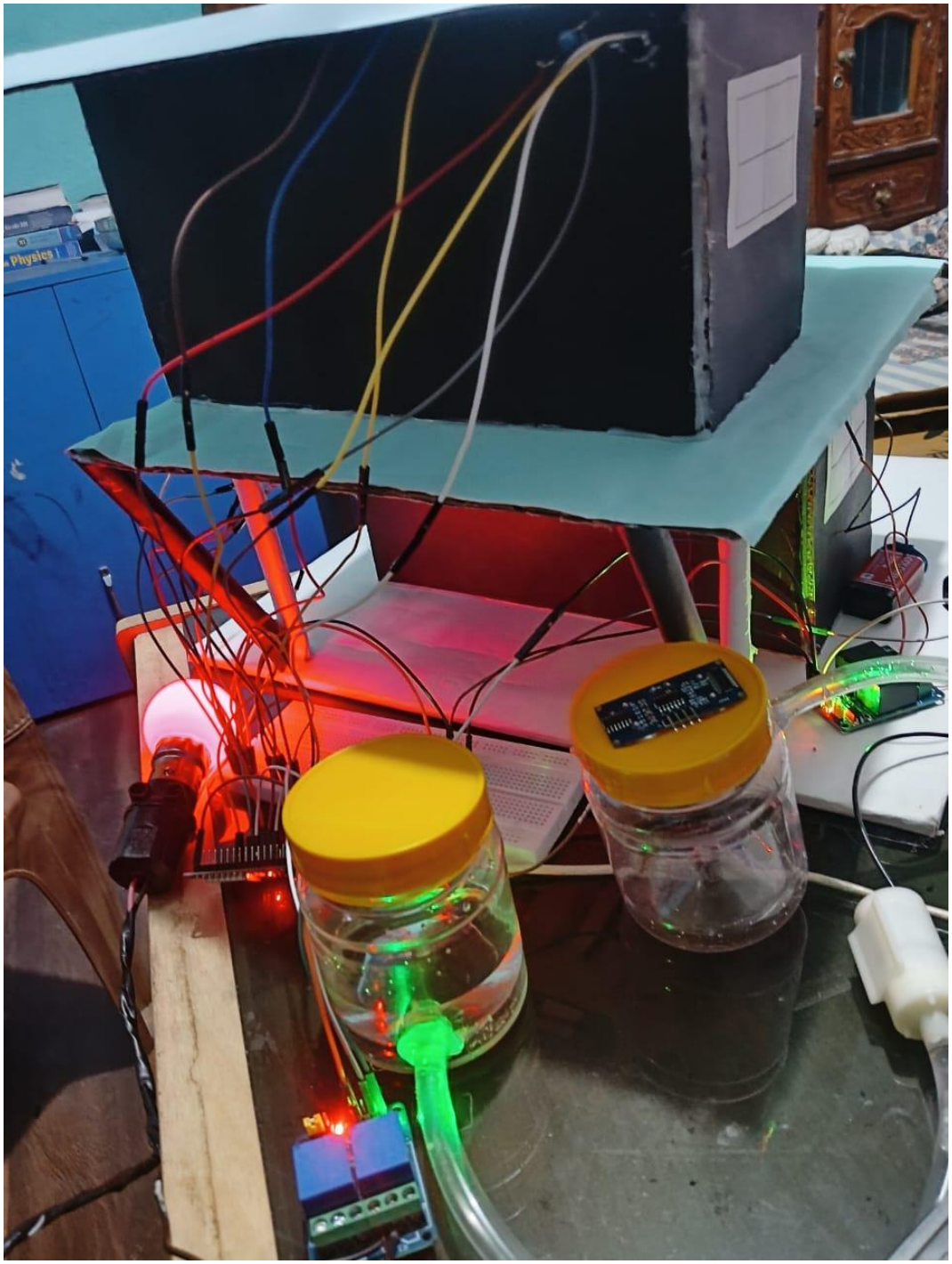

Figure 55:Final Product from back POV

Appendix C:Design Diagram

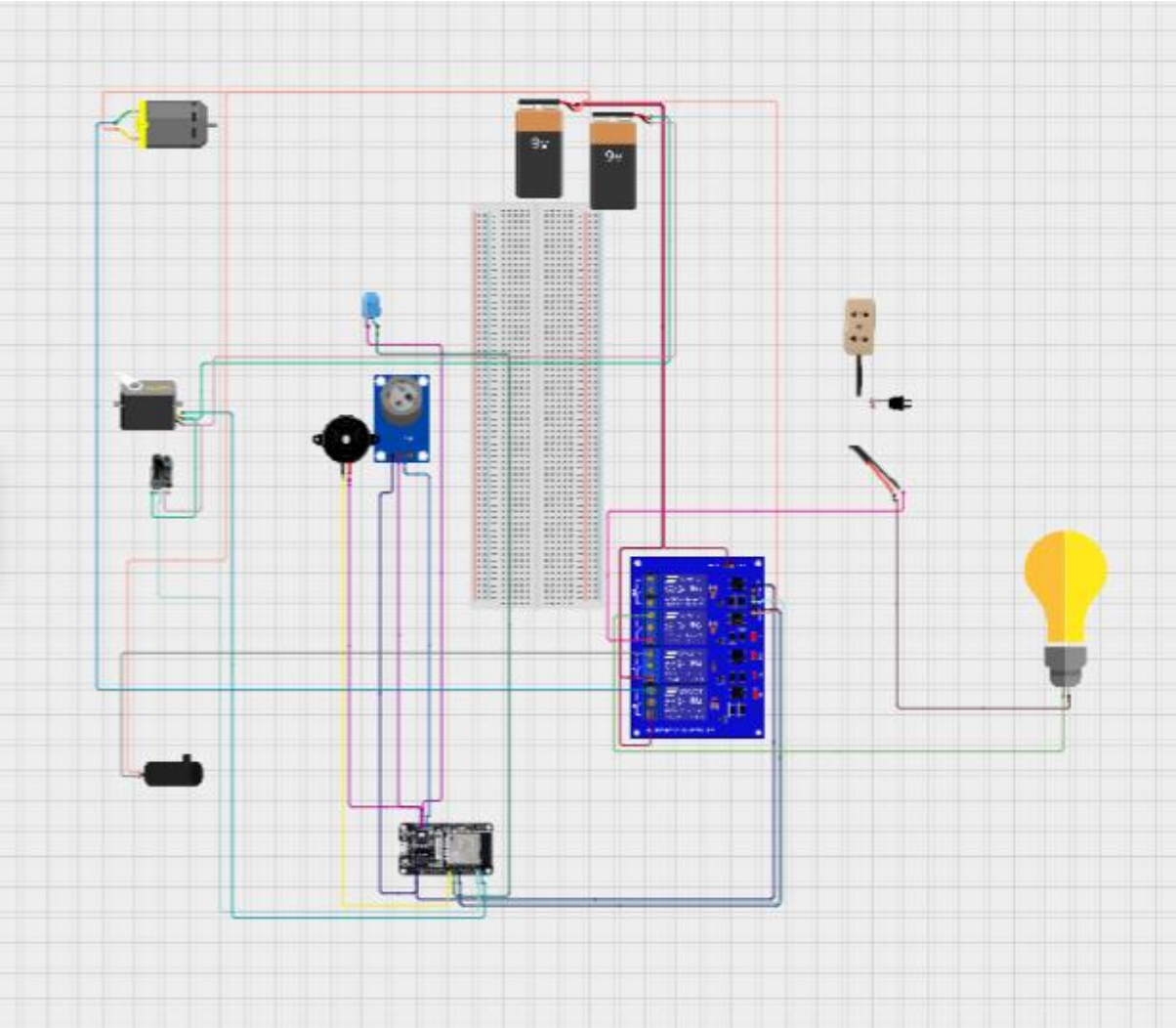

Figure 56:Design Diagram

8.3 Appendix C: Others

8.3.1 Meeting Log

8.3.1.1 Logbook 1

Group .COM Logbook

Meeting No: 01 **Date: 22/11/2025**

Start Time: 1:15 pm **Finish Time: 2:00 pm**

Items Discussed:

- Group was formed.
- Different project ideas were discussed.
- Task and role division among the members for developing the project.
- Discussion of the required sensors and components for the home automation project and fund collection.

Achievements:

- Project objectives were clearly understood.
- Smart home project was finalized.
- Required sensors and components were finalized.
- Group roles and tasks were successfully assigned.

Problems:

Difficulty while choosing the sensors due to limited understanding of the sensors.

Tasks for Next Meeting:

- Study the sensor working principles.
- Collect all the required components.

Student's Signature

Teacher's Signature

Figure 57 LogBook-1

8.3.1.2 **Logbook 2**

Group .COM Logbook

Meeting No: 02 **Date: 15/12/2025**

Start Time: 1:15 pm **Finish Time: 2:00 pm**

Items Discussed:

- Basic structure of home prototype.
- Esp32 setup, breadboard connections, and Arduino IDE configuration.
- Initial testing of PIR sensor for motion detection.

Achievements:

Student's Name	Achievement
Nischal Rai	Studied about the different sensors and collected the hardware components required for the project.
Sarobar Pokhrel	Installed all the necessary drivers and components required to connect esp32 and uploaded the code in IDE.
Suranga Rai	Prepared and designed the home prototype layout.
Saphal Bohora	Brought the required hardware components.
Nimesh Tamang	Observed the testing process of PIR and results for documenting.

Problems:

- PIR sensor sensitivity varied due to environmental factors.
- Sensor did not work properly due to loose wire connections.

Tasks for Next Meeting:

- Test all the sensors used in the project.
- Start assembling the complete home prototype.

Student's Signature Teacher's Signature

Figure 58 LogBook-2

8.3.1.3 **Logbook 3**

Group .COM Logbook

Meeting No: 03 **Date: 01/01/2026**

Start Time: 2:00pm **Finish Time: 3:00pm**

Items Discussed:

- Finalization of the prototype home model.
- Testing of all sensors on the prototype.
- Preparation of documentation for milestone submission.

Achievements:

Student's Name	Achievement
Nischal Rai	Installed all required sensors on prototype house and checked its connections.
Sarobar Pokhrel	Tested all installed sensors like PIR, MQ-2 and others using code installed in ESP32
Suranga Rai	Assembled and built the prototype house model.
Saphal Bohora	Captured all required evidence of full system testing.
Nimesh Tamang	Prepared documentation including working sensor, system testing and prototype model.

Problems:

- Insufficient materials while building prototype house model.
- Difficulty while installing sensor on prototype house model

Tasks for Next Meeting:

- Replace PIR Sensor with IR Sensor and Adjust prototype layout and update code for IR Sensor
- Analyze, observed PIR Sensor issue and record in documentation

Student's Signature Teacher's Signature

Figure 59 LogBook-3

8.3.1.4 **Logbook 4**

Group .COM Logbook

Meeting No: 04 **Date: 01/10/2026**

Start Time: 5:00pm **Finish Time: 5:30pm**

Items Discussed:

- Rewired PIR Sensor System and replace with IR Sensor
- To support newly added IR Sensor, code is updated and retested system
- Adjust prototype layout for newly added IR Sensor, Captured Working evidence of complete system and finalized documentation for submission

Achievements:

Student's Name	Student's Achievement
Nischal Rai	IR Sensor was added by replacing and rewiring PIR Sensor System
Sarobar Pokhrel	Updated code to support newly added IR Sensor and retested system
Suranga Rai	Prototype model was adjusted for IR Sensor placement by replacing PIR Sensor
Saphal Bohora	Noted and captured final working evidence of complete system
Nimesh Tamang	Saw and analyzed final working evidence and finalized document for submission

Problems:

- Difficult to modify PIR Sensor as it was already implemented
- Difficult to make place in prototype for IR Sensor by replacing PIR Sensor
- Difficult to replace PIR Sensor with IR Sensor and updated code to support IR Sensor

Tasks for Next Meeting

- Finalized of project and reviewed completed document

Student's Signature Teacher's Signature

Figure 60 LogBook-4

8.3.1.5 **Logbook 5**

Group .COM Logbook

Meeting No: 05 **Date: 14/01/2026**

Start Time: 8:00am **Finish Time: 4:00pm**

Items Discussed:

- Check completed prototype model system whether it is working properly or not.
- Review finalized documentation.

Achievements:

Student's Name	Student's Achievement
Nischal Rai	Reviewed finalized document if any topic is missing or not and observed if prototype model is working or not
Sarobar Pokhrel	Observed if any component is missing or not in Prototype model System and reviewed final documentation
Suranga Rai	Reviewed final proposal documentation and reviewed final documentation
Saphal Bohora	Reviewed documentation and observed if prototype model is working or not
Nimesh Tamang	Missed topic written and updated in document and observed if prototype model is working or not

Problems:

- Small topic missed and updated in document

Tasks for Next Meeting:

- Submit final proposal and final documentation.

Student's Signature Teacher's Signature

Figure 61 LogBook-5

8.3.2 Future work

In the future, the system can be improved in a number of ways. Remote monitoring and control can be done through mobile application integration. Data analysis and storage can be facilitated by cloud connection. The sensors can be added like the temperature and humidity sensors. It can be further enhanced by voice control or voice assistance and automation through AI too. These will ensure that the system is smarter, more scalable and friendly to the user.